

High-Performance E-Commerce Backend Engine

Structured Backend Report for Requirements 1-5

Course: Parallel Programming **Project:** High-Performance E-Commerce Backend Engine **Stack:** Django, Django REST Framework, PostgreSQL, PgBouncer, Redis, Celery, Nginx, Gunicorn, Docker Compose, Locust **Report Scope:** Requirements 1, 2, 3, 4, and 5

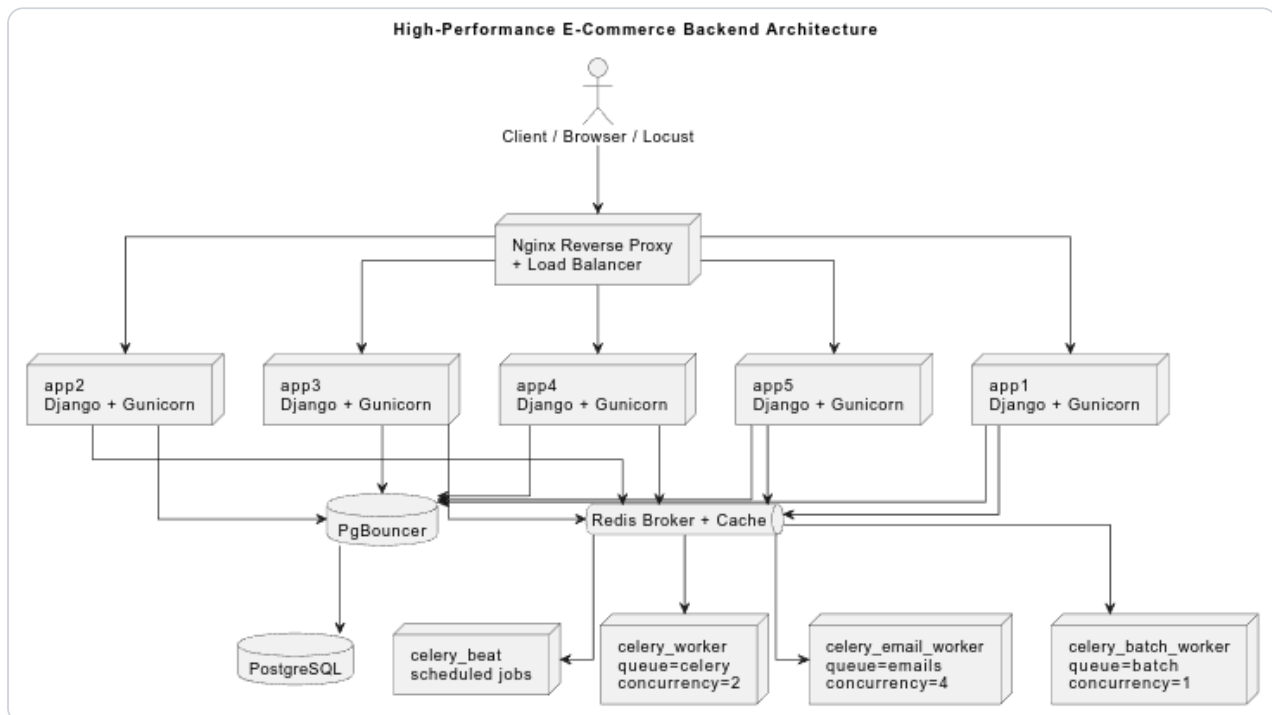
1. Executive Summary

This project implements a production-style e-commerce backend that handles concurrent checkout, payment processing, wallet balance updates, product reservation, background tasks, batch jobs, and load distribution.

The system runs five Django/Gunicorn application containers behind Nginx. PostgreSQL stores persistent data, PgBouncer controls database connection pressure, Redis is used for cache and Celery brokering, Celery workers process asynchronous jobs, and Locust validates behavior under concurrent load.

Requirement	Topic	Implemented Solution	Evidence
Requirement 1	Race condition safety	Five concurrency cases using row locks and state checks	<code>race1_before</code> vs <code>race1_after</code> Locust modes
Requirement 2	Resource management	Gunicorn gthread workers, PgBouncer pooling, bounded Celery concurrency	5 app containers x 8 workers x 3 threads = 120 request handlers
Requirement 3	Asynchronous queues	Redis + Celery with separated <code>emails</code> , <code>batch</code> , and <code>celery</code> queues	Sync email/payment waits; async path returns immediately
Requirement 4	Batch processing	Daily sales aggregation in chunks on a dedicated batch worker	Naive task loads all orders; optimized task logs chunks
Requirement 5	Load distribution	Nginx Least Connections across <code>app1</code> , <code>app2</code> , <code>app3</code> , <code>app4</code> , and <code>app5</code>	Nginx upstream logs show all five backends

2. Current System Architecture



Main runtime containers used by the backend:

Container	Purpose
<code>ecommerce_nginx</code>	Public HTTP entry point and load balancer
<code>ecommerce_app1</code> to <code>ecommerce_app5</code>	Django/Gunicorn API workers
<code>ecommerce_db</code>	PostgreSQL database
<code>ecommerce_pgouncer</code>	Transaction-pooling layer in front of PostgreSQL
<code>ecommerce_redis</code>	Redis cache and Celery broker
<code>ecommerce_celery_worker</code>	General Celery queue
<code>ecommerce_celery_email_worker</code>	Email/background notification queue
<code>ecommerce_celery_batch_worker</code>	Batch-processing queue
<code>ecommerce_celery_beat</code>	Periodic task scheduler

2.1 Implemented Functional Requirements

This section summarizes the main functional requirements already implemented by the backend before the report focuses on the concurrency, capacity, queueing, batch-processing, and load-distribution requirements.

User Management

1. Users can register and log in.
2. Users receive an authentication token after login.
3. Customers can access shopping, cart, and order features.
4. Admin users have elevated permissions for product and order management.
5. Admins cannot use customer-only flows like cart checkout or product reservation.

Product Management

1. Customers can view active products.
2. Customers can view product details.
3. Admins can create products.
4. Admins can update products.
5. Admins can delete or deactivate products.
6. Admins can restock products.
7. Admins can view inventory logs.
8. Product stock is tracked.

9. Product stock changes are logged.

Cart Management

1. Customers can view their cart.
2. Customers can add products to cart.
3. Customers can update cart item quantity.
4. Customers can remove products from cart.
5. Customers can clear the cart.
6. Cart operations validate product existence and stock.
7. Admins are blocked from using carts.

Order Management

1. Customers can checkout their cart.
2. Checkout creates an order from cart items.
3. Checkout deducts product stock.
4. Checkout creates order items.
5. Checkout creates payment records.
6. Customers can view their own orders.
7. Customers can view details of their own orders.
8. Admins can view all orders.
9. Customers can cancel their own valid orders.
10. Admins can update order status.
11. Invalid order state transitions are blocked.

Payment

1. Customers can process payment for their own order.
2. Payment status can be marked completed.
3. Duplicate payment processing is blocked.
4. Payment cannot be processed for cancelled orders.
5. Wallet checkout is supported.
6. Wallet balance is checked before purchase.
7. Wallet balance is deducted after successful wallet checkout.
8. Insufficient wallet balance blocks checkout.

Product Reservation

1. Customers can reserve product stock temporarily.
2. Product reservation creates an `OrderLock` .
3. Expired locks can be released.
4. Over-reservation is prevented by row locking.
5. Admins cannot reserve products.

Reviews

1. Customers can view product reviews.
2. Customers can create product reviews.
3. A customer can only review a product they purchased.
4. A customer can only submit one review per product.

Async / Background Processing

1. Order confirmation email is sent asynchronously.
2. Order cancellation email is sent asynchronously.
3. Wallet payment can be processed asynchronously through Celery.
4. Daily sales batch processing is supported.
5. Batch processing supports chunked processing.
6. Expired product reservation locks can be cleaned periodically.

Concurrency and Data Consistency Requirements

1. Checkout must prevent overselling.
 2. Wallet checkout must prevent double spending.
 3. Payment processing must prevent duplicate payment.
 4. Cancel/payment race must preserve valid order state.
 5. Product reservation must prevent over-reservation.
-

3. Requirement 1 - Race Condition Safety

Overview

The implemented backend is not only a simple CRUD API. It includes explicit concurrency-control demonstrations for five race-condition cases:

1. Concurrent Checkout / Overselling
2. Wallet Checkout / Double Spend
3. Double Payment Processing
4. Cancel Order While Payment Is Processing
5. Product Reservation / Over-Reservation

3.1 Goal

The goal is to prove that shared e-commerce data can break under concurrent access, then show how the implemented backend prevents those failures.

The project validates five concrete race-condition cases, not only a single stock race.

Case	Problem	Demo / Risk Endpoint	Protected Endpoint
Case 1	Concurrent checkout can oversell stock	<code>POST /api/orders/race-demo/</code>	<code>POST /api/orders/checkout/</code>
Case 2	Wallet checkout can double-spend balance	<code>POST /api/orders/blocking-wallet-checkout/</code>	<code>POST /api/orders/checkout-wallet-async/</code>
Case 3	Same order can be paid twice	<code>POST /api/orders/<id>/process-payment-unsafe/</code>	<code>POST /api/orders/<id>/process-payment/</code>
Case 4	Paid order can be cancelled during payment transition	<code>POST /api/orders/<id>/cancel-unsafe/</code>	<code>POST /api/orders/<id>/cancel/</code>
Case 5	Product reservations can exceed available stock	<code>POST /api/products/<id>/reserve-unsafe/</code>	<code>POST /api/products/<id>/reserve/</code>

3.2 Synchronization Techniques Used

Technique	Used In	Reason
<code>transaction.atomic()</code>	Checkout, wallet checkout, payment, cancel, restock, reservation	Keeps related writes all-or-nothing
<code>select_for_update()</code>	Product rows, user wallet row, order row, payment row	Serializes critical updates on shared rows
Consistent lock ordering	Product rows sorted by id during checkout	Reduces deadlock risk
State validation	Payment and cancellation endpoints	Prevents invalid transitions such as paying a cancelled order
Optimistic version field	Product helper logic	Detects lower-risk inventory conflicts without holding locks

3.3 Locking Implementation Details

The backend uses two locking strategies depending on the risk level of the operation.

Optimistic Locking

Optimistic locking is used where conflicts are rare and retrying the operation is safe.

1. `apps/cart/models.py` - `CartItem.version` field

The `version` integer field on the `CartItem` table is the foundation. Every update increments it. If two requests read `version=3`, both try to update with `WHERE version=3`. Only the first succeeds with one affected row. The second receives zero affected rows, so the conflict is detected.

2. `apps/products/models.py` - `Product.version` field

The same mechanism exists on the `Product` model. Every stock change bumps the version number.

3. `apps/cart/services.py` - `_try_add_to_cart_optimistic()` and `add_to_cart()`

When adding to cart, the service reads the `CartItem` without taking a database lock, then tries to update with `WHERE version=<captured_version>`. If zero rows are updated, the helper returns a conflict signal. `add_to_cart()` retries up to five times with exponential back-off using `0.03s * attempt`.

4. `apps/products/services.py` - `deduct_stock_optimistic()` and `deduct_stock_with_retry()`

The pre-checkout stock check reads the product without a lock, then attempts `UPDATE WHERE version=<captured_version> AND stock >= quantity`. Zero affected rows means either a version conflict or insufficient stock. The retry wrapper retries up to five times. This is used for cart adds and validations, not the final checkout commit.

Pessimistic Locking (`SELECT FOR UPDATE`)

Pessimistic locking is used where conflicts are likely or the consequences are irreversible.

5. `apps/orders/services.py` - `create_order_from_cart()`

Locks all product rows at once with `select_for_update()`, ordered by ID to reduce deadlock risk. Any other checkout touching the same products blocks until the transaction commits.

6. `apps/orders/services.py` - `checkout_with_wallet()`

Locks the user row first to protect wallet balance from double-spend across multiple tabs, then locks all product rows. This prevents a user from spending more than their available balance.

7. `apps/orders/services.py` - `process_payment()`

Locks both the `Order` row and the `Payment` row. This prevents duplicate payment processing, which is a financial error that should not be allowed.

8. `apps/orders/services.py` - `cancel_order()`

Locks the order row to prevent a race between a cancel request and a payment-processing request hitting the same order at the same time.

9. `apps/orders/services.py` - `update_order_status()`

Admin status updates lock the order row so two concurrent admin updates do not produce an inconsistent final state.

10. `apps/cart/services.py` - `update_cart_item_quantity()`

Setting quantity to exactly `N` requires strict serialization. Pessimistic locking is used so the final result is unambiguous.

11. `apps/products/services.py` - `restock_product()`

Admin restock locks the product row so two simultaneous restocks of the same product cannot corrupt the total stock count.

12. `apps/products/services.py` - `create_order_lock()`

Temporary stock reservation during checkout locks the product row exclusively while creating the reservation record.

13. `apps/products/services.py` - `deduct_stock_pessimistic()`

Called from checkout after the product row is already locked. It safely deducts stock and writes an `InventoryLog` audit entry under that lock.

`transaction.atomic()` ACID Wrapper

Every function above is also wrapped in `transaction.atomic()`, which means related writes commit together or roll back together:

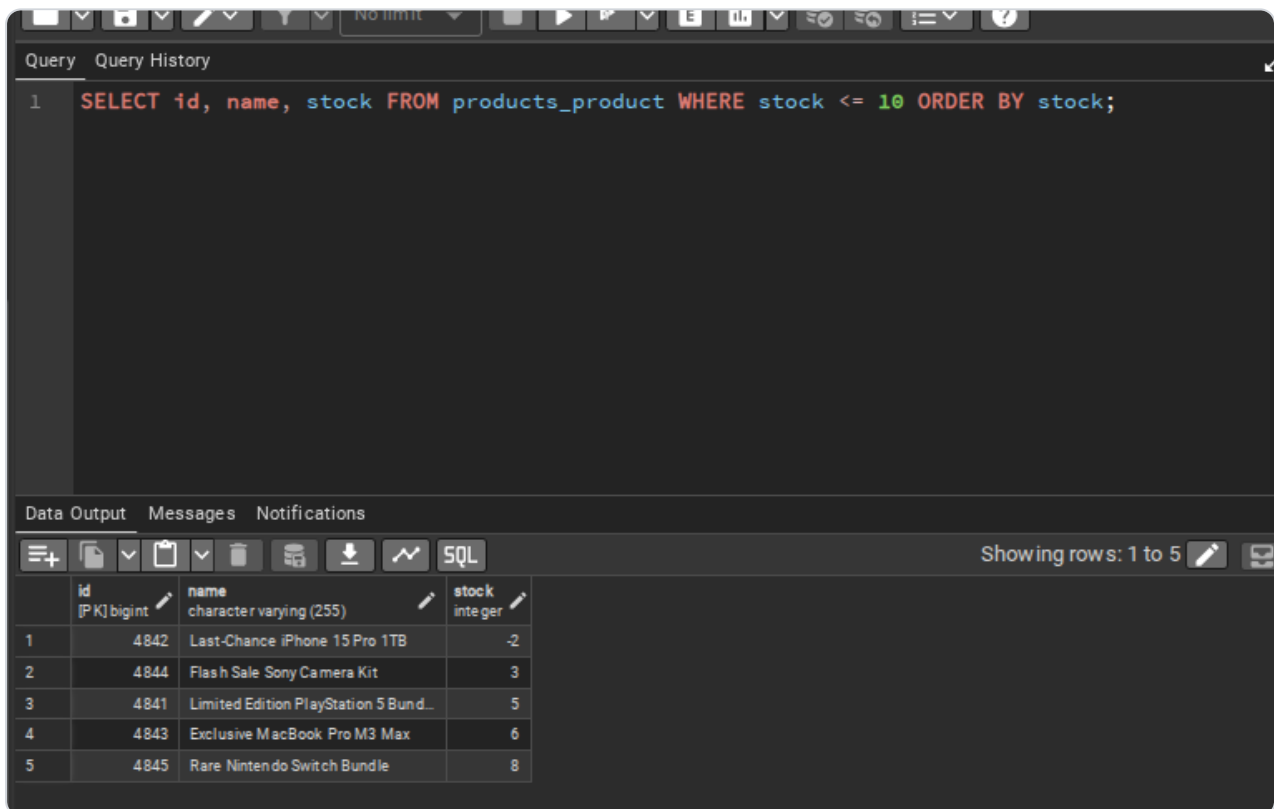
1. `create_order_from_cart()` - stock deduction, order creation, and payment record creation all succeed or all roll back.
2. `checkout_with_wallet()` - wallet deduction, stock deduction, order creation, and payment creation are one atomic unit.
3. `process_payment()` - payment status and order status are never partially updated.
4. `cancel_order()` - cancellation is committed as one clean state transition.
5. `deduct_stock_pessimistic()` - stock change and `InventoryLog` audit entry are written together, so the audit trail stays in sync.

3.4 Case 1: Concurrent Checkout / Overselling

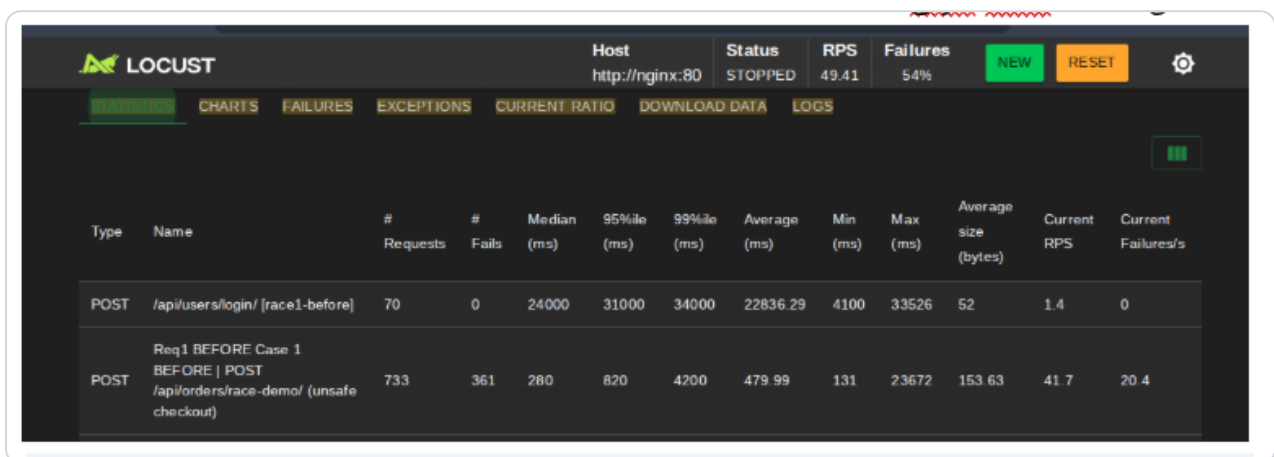
Explanation of the first case: This issue occurs when multiple users attempt to purchase the same low-stock product simultaneously.

Before resolution, the unsafe checkout endpoint reads the stock value without locking the product record. Possible outcome: Inventory could become negatively valued, which proves that overselling has occurred.

this db before start test:



now we start test: This is a Python-based testing tool called locust that allows us to synchronize a selected number of requests.



Before: POST /api/orders/race-demo/

The unsafe endpoint intentionally reads a product without locking, sleeps for 100 ms, then decrements stock using `F("stock") - 1` without enforcing a stock floor. Under concurrent users, many requests can proceed from stale stock knowledge and push stock below zero.

and this fail of endpoint

```
1 POST Req1 BEFORE Case 1 BEFORE | POST /api/orders/race- CatchResponseError("OVERSELL: stock=-1 product=2443")
demo/ (unsafe checkout)
```

and this db after start test (We have now examined the database, checked the inventory, and confirmed that the problem does indeed exist.):

id	name	stock
1	Limited Edition PlayStation 5 Bund...	-2194
2	Last-Chance iPhone 15 Pro 1TB	-48
3	Exclusive MacBook Pro M3 Max	-1778
4	Flash Sale Sony Camera Kit	-1221
5	Rare Nintendo Switch Bundle	-52
306	Limited Edition PlayStation 5 Bund...	-1
307	Last-Chance iPhone 15 Pro 1TB	7
308	Exclusive MacBook Pro M3 Max	-5084
309	Flash Sale Sony Camera Kit	4
310	Rare Nintendo Switch Bundle	3
611	Limited Edition PlayStation 5 Bund...	3
612	Last-Chance iPhone 15 Pro 1TB	2
613	Exclusive MacBook Pro M3 Max	-2821
614	Flash Sale Sony Camera Kit	7
615	Rare Nintendo Switch Bundle	3
916	Limited Edition PlayStation 5 Bund...	4
917	Last-Chance iPhone 15 Pro 1TB	7
918	Exclusive MacBook Pro M3 Max	-145
919	Flash Sale Sony Camera Kit	4
920	Rare Nintendo Switch Bundle	3
1221	Limited Edition PlayStation 5 Bund...	6
1222	Last-Chance iPhone 15 Pro 1TB	8

After: Solution Now we will discuss how to solve this type of problem:

1. Order Creation / Preventing Overselling Method Used: Pessimistic Locking + `transaction.atomic()`

Pessimistic Locking was used because the order creation process modifies inventory, a shared resource among multiple users. Therefore, product records are locked using `select_for_update()` before inventory is checked and quantities are deducted.

`Transaction.atomic()` was also used to ensure that the order creation process either completes fully or fails completely.

Safe Outcome: Only the available quantity is sold, any overselling order is rejected, and inventory cannot become negative.

The real checkout path calls `create_order_from_cart()`. It starts a transaction, locks all product rows with `select_for_update()`, validates stock while the locks are held, deducts stock, creates the order, creates order items, creates the payment row, clears the cart, and invalidates the product cache.

Expected safe result:

```
Stock never becomes negative.
Some requests succeed with HTTP 201.
Excess requests are rejected with HTTP 400 when stock is exhausted.
```

test after Solution with locust:

LOCUST

Host

http://nginx:80

Status

STOPPED

RPS

35.1

Failures

15%

NEW

RESET

STATISTICS

CHARTS

FAILURES

EXCEPTIONS

CURRENT RATIO

DOWNLOAD DATA

LOGS

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/api/users/login/ [race1-after]	79	0	28000	37000	39000	25109.36	1769	39415	52	3.5	0
POST	Req1 AFTER Case 1 AFTER POST /api/orders/checkout/ (locked checkout)	257	0	550	4300	13000	986.07	78	14032	418.35	11.7	0
POST	Req1 AFTER Case 2 POST /api/orders/checkout-wallet-async/ (wallet double-spend protected)	173	0	220	3200	5500	568.32	15	6993	421.5	8.5	0
POST	Req1 AFTER Case 3 double payment POST /api/orders/{id}/process-payment/	309	0	240	610	1100	346.18	13	13934	79.63	16.1	0
POST	Req1 AFTER Case 4 cancel while payment processing POST /api/orders/{id}/cancel/	65	0	200	470	2700	259.39	17	2738	53	2.7	0
POST	Req1 AFTER Case 5 POST /api/products/{id}/reserve/ (over-reservation protected)	191	0	190	530	2100	302.81	13	14539	40.14	8.6	0
POST	Req1 AFTER Setup POST /api/cart/add/	712	268	260	740	1400	379.29	12	23061	142.19	33.3	13.6
	Aggregated	1786	268	270	4700	31000	1560.52	12	39415	180.01	84.4	13.6

ABOUT

ABOUT

this fail (because the stock empty and the logic return 404 not found)

 LOCUST

Host

http://nginx:80

Status

STOPPED

RPS

53.11

Failures

2%

NEW

RESET



STATISTICS

CHARTS

FAILURES

EXCEPTIONS

CURRENT RATIO

DOWNLOAD DATA

LOGS

#

Method

Name

Message

Failures

194

POST

/api/cart/add/

CatchResponseError('Cart add failed: 404')

and this from our terminal:

```
=====
REQUIREMENTS TEST REPORT
=====

Mode: race1_after

REQ 1 – Race Condition Safety (5 Cases)
  [General] HOT checkouts via EcommerceUser succeeded : 0
  [General] Stock-outs blocked                        : 0

CASE 1 – Concurrent Checkout / Overselling
  BEFORE endpoint : POST /api/orders/race-demo/ (no lock, 100ms sleep → stock < 0)
  AFTER  endpoint : POST /api/orders/checkout/  (SELECT FOR UPDATE on product rows)
  Oversell events (stock<0)      : 0
  Safe checkout ok (201)         : 237
  Correctly blocked (400)        : 20
  ✓ PROTECTED – pessimistic lock prevented overselling
```

no negative in Db :

```
o [eabraheem@arch high-performance-ecommerce-backend]$ docker exec ecommerce_db psql -U ecommerce_user -d ecommerce_db -c "SELECT id, name, stock FROM products_product WHERE stock <= 10 ORDER BY stock;"
\
 id | name | stock
-----+-----+-----
13686 | Limited Edition PlayStation 5 Bundle | 0
13689 | Flash Sale Sony Camera Kit | 0
13690 | Rare Nintendo Switch Bundle | 0
13991 | Limited Edition PlayStation 5 Bundle | 0
13992 | Last-Chance iPhone 15 Pro 1TB | 0
13995 | Rare Nintendo Switch Bundle | 0
13687 | Last-Chance iPhone 15 Pro 1TB | 0
13994 | Flash Sale Sony Camera Kit | 0
13993 | Exclusive MacBook Pro M3 Max | 0
13688 | Exclusive MacBook Pro M3 Max | 0
(10 rows)

> |
```

3.5 Case 2: Wallet Checkout / Double Spend

This occurs when multiple payment requests are sent simultaneously from the same user's wallet account.

Before a solution is found, if the wallet balance is not protected, two requests might read the same balance before the amount is debited.

Potential outcome: The user could potentially withdraw the same balance more than once.

Example: If the balance is 100, and two requests totaling 80 are sent simultaneously, both requests might succeed without proper protection. **Demo/Risk path:** `POST /api/orders/blocking-wallet-checkout(double-spend)`

This endpoint maps to the synchronous wallet checkout. It demonstrates the risk area: several concurrent wallet purchases compete for the same user balance while the payment gateway simulation takes 3 seconds. In the current code, the service already protects this path by locking the user row before checking and deducting wallet balance.

now the test in locust: This image was taken from a test of the Wallet Checkout/Double Spend in Locust scenario, where 269 simultaneous requests were sent to the wallet payment endpoint. It shows an insufficient number of requests that failed or were rejected, which is expected in this scenario because the system prevents multiple attempts to allocate phone credit during a simultaneous push.

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/api/users/login/ [race1-before]	70	0	24000	31000	34000	22836.29	4100	33526	52	1.4	0
POST	Req1 BEFORE Case 1 BEFORE POST /api/orders/race-demo/ (unsafe checkout)	733	361	280	820	4200	479.99	131	23672	153.63	41.7	20.4
POST	Req1 BEFORE Case 2 POST /api/orders/blocking-wallet-checkout/ (wallet double-spend)	269	234	150	3400	3800	634.12	11	4094	190.96	16	13.5

This image shows that Locust recorded 234 rejected requests for Wallet Checkout/Double Spend, due to the appearance of code 409 Conflict. Code 409 here does not indicate a system failure, but rather that the system detected a conflict attempt during the wallet payment and blocked it securely. This means that another request was attempting to use the same wallet balance at the same time, so the request was rejected to protect the balance and prevent a double-spend issue.

234	POST	Req1 BEFORE Case 2 POST /api/orders/blocking-wallet-checkout/ (wallet double-spend)	CatchResponseError('409 Conflict: wallet double-spend race blocked')
-----	------	---	--

This analysis is performed at the terminal after the test is complete.

```
CASE 2 – Wallet Checkout / Double Spend
BEFORE endpoint : POST /api/orders/blocking-wallet-checkout/
AFTER  endpoint : POST /api/orders/checkout-wallet-async/
Scenario : concurrent wallet checkouts – user row locked to prevent double spend
Queued (202)           : 35
Blocked – low balance (400) : 234
```

We took a screenshot of the balance for two reasons:

1. To prove that simultaneous orders put pressure on the balance (meaning that many purchases were made from the wallet):

Query: `SELECT email, wallet_balance FROM users_user WHERE email LIKE 'locust_@test.com' ORDER BY wallet_balance ASC LIMIT 10;`

	email character varying (255)	wallet_balance numeric (10,2)
1	locust_79@test.com	1.61
2	locust_77@test.com	9.09
3	locust_5@test.com	29.19
4	locust_6@test.com	40.28
5	locust_95@test.com	43.00
6	locust_40@test.com	54.17
7	locust_72@test.com	61.97
8	locust_100@test.com	66.35
9	locust_86@test.com	67.45
10	locust_3@test.com	73.74

1. Proof that the system did not allow double spend (there is no negative balance, meaning the user did not spend more than their balance).
- 2.

Query: `SELECT COUNT(*) AS negative_wallets FROM users_user WHERE wallet_balance < 0;`

	negative_wallets bigint
1	0

After: `POST /api/orders/checkout-wallet-async/`

Short Professional Summary:

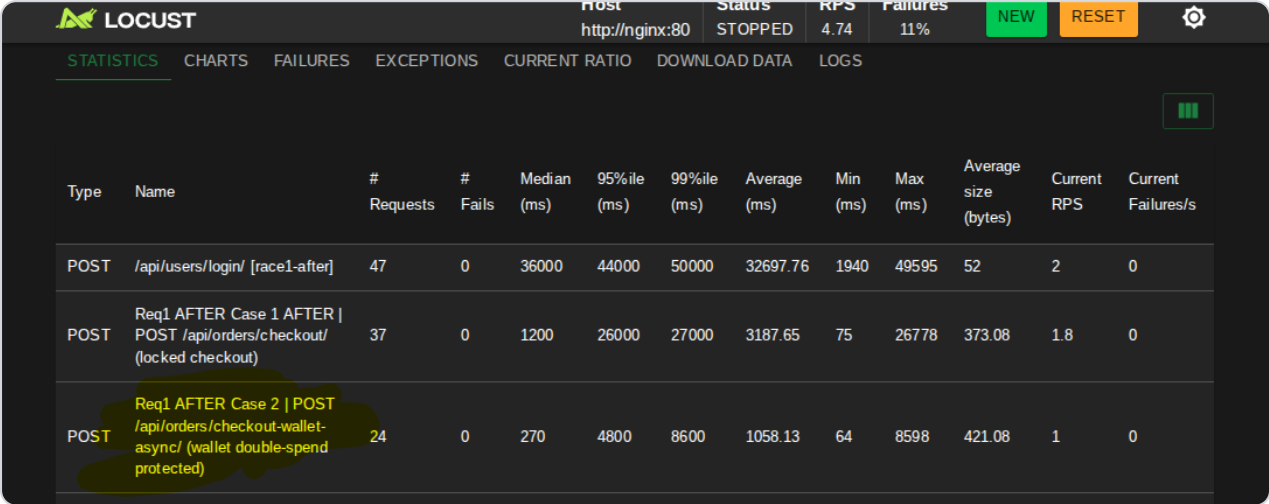
The wallet checkout is handled asynchronously to improve response time. The endpoint immediately returns HTTP 202 Accepted and queues `process_wallet_payment_async` to process the payment in the background.

The actual wallet operation runs inside `checkout_with_wallet()` using `transaction.atomic()`. It locks the user row with `select_for_update()` before checking and deducting the wallet balance, and also locks product rows before updating stock.

This ensures that concurrent requests cannot use the same wallet balance or modify the same stock at the same time.

Safe Result:

The first valid wallet checkout succeeds, while duplicated or concurrent attempts are safely rejected if the balance or stock is no longer sufficient. This prevents double spending and keeps wallet, stock, and payment data consistent.



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/api/users/login/ [race1-after]	47	0	36000	44000	50000	32697.76	1940	49595	52	2	0
POST	Req1 AFTER Case 1 AFTER POST /api/orders/checkout/ (locked checkout)	37	0	1200	26000	27000	3187.65	75	26778	373.08	1.8	0
POST	Req1 AFTER Case 2 POST /api/orders/checkout-wallet-async/ (wallet double-spend protected)	24	0	270	4800	8600	1058.13	64	8598	421.08	1	0

3.6 Case 3: Double Payment Processing

This situation occurs when a payment processing request is sent for the same payment more than once.

Before resolution, if the payment history is not locked, two requests might see the payment status as "pending."

Potential outcome:

The same payment might be processed twice.

In the current system, a new payment queue is not created for each attempt, but the risk arises when an unsafe endpoint accepts the second payment attempt and modifies or replaces the transaction ID.

Before: `POST /api/orders/<id>/process-payment-unsafe/`

The unsafe endpoint reads the order and payment without row locks. It sleeps for 100 ms, then marks the payment as completed even if another request already completed it. This simulates a duplicate charge bug.

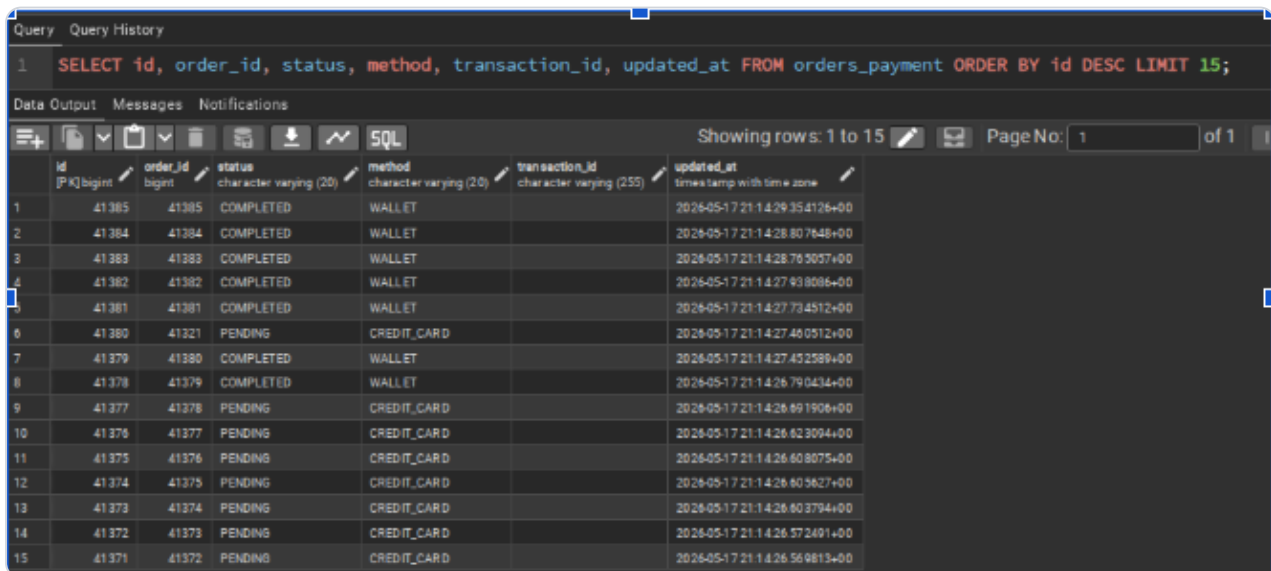
Repeated payment requests were sent for the same application using an insecure endpoint. The number of failures in Locust does not indicate server failure, but rather that Locust detected that the duplicate payment was accepted, which constitutes a race-condition conflict.

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/api/users/login/ [race]-before]	70	0	24000	31000	34000	22836.29	4100	33526	52	1.4	0
POST	Req1 BEFORE Case 1 BEFORE POST /api/orders/race-demo/ (unsafe checkout)	733	361	280	820	4200	479.99	131	23672	153.63	41.7	20.4
POST	Req1 BEFORE Case 2 POST /api/orders/blocking-wallet-checkout/ (wallet double-spend)	269	234	150	3400	3800	634.12	11	4094	190.96	16	13.5
POST	Req1 BEFORE Case 3 double payment POST /api/orders/{id}/process-payment-unsafe/	468	230	240	600	1300	310.91	117	10244	214.51	25.9	13

This message proves that the same order was paid more than once through an insecure endpoint. Locust registered it as a conflict because repeated payment for the same order is a serious error.

1	POST	Req1 BEFORE Case 3 double payment POST /api/orders/{id}/process-payment-unsafe/	CatchResponseError('409 Conflict: duplicate payment processed for order=40722')
---	------	---	---

this db It appears that some payments have been completed. Since the system uses a one-to-one relationship between the order and the payment, a second payment row for the same order will not appear. Therefore, we are not looking for two rows, but rather for the same payment row to have been updated again.



The screenshot shows a database query result in a tool with a dark theme. The query is: `SELECT id, order_id, status, method, transaction_id, updated_at FROM orders_payment ORDER BY id DESC LIMIT 15;`. The results are displayed in a table with 7 columns: `id` (PK bigint), `order_id` (bigint), `status` (character varying (20)), `method` (character varying (20)), `transaction_id` (character varying (255)), and `updated_at` (timestamp with time zone). The table shows 15 rows of data, with the first 5 rows having a status of 'COMPLETED' and the remaining 10 rows having a status of 'PENDING'.

id	order_id	status	method	transaction_id	updated_at
41385	41385	COMPLETED	WALLET		2026-05-17 21:14:29.354126+00
41384	41384	COMPLETED	WALLET		2026-05-17 21:14:28.807648+00
41383	41383	COMPLETED	WALLET		2026-05-17 21:14:28.765057+00
41382	41382	COMPLETED	WALLET		2026-05-17 21:14:27.938086+00
41381	41381	COMPLETED	WALLET		2026-05-17 21:14:27.734512+00
41380	41321	PENDING	CREDIT_CARD		2026-05-17 21:14:27.460512+00
41379	41380	COMPLETED	WALLET		2026-05-17 21:14:27.432589+00
41378	41379	COMPLETED	WALLET		2026-05-17 21:14:26.790434+00
41377	41378	PENDING	CREDIT_CARD		2026-05-17 21:14:26.691908+00
41376	41377	PENDING	CREDIT_CARD		2026-05-17 21:14:26.623094+00
41375	41376	PENDING	CREDIT_CARD		2026-05-17 21:14:26.608075+00
41374	41375	PENDING	CREDIT_CARD		2026-05-17 21:14:26.605827+00
41373	41374	PENDING	CREDIT_CARD		2026-05-17 21:14:26.603794+00
41372	41373	PENDING	CREDIT_CARD		2026-05-17 21:14:26.572491+00
41371	41372	PENDING	CREDIT_CARD		2026-05-17 21:14:26.569813+00

The image illustrates the database state during the third state test of the first requirement, where purchase orders associated with payment transactions are examined. It shows that some orders have reached the "Completed" payment state while others remain "Pending," which helps demonstrate that a concurrent processing issue occurred before the protection was applied, such as the possibility of executing the payment or attempting to update the payment state asynchronously.

Query

Query History

1 AS payment_status, p.transaction_id FROM orders_order o JOIN orders_payment p ON p.order_id = o.order_id

Data Output

Messages

Notifications

Showing rows: 1 to 15

	order_id bigint	order_status character varying (20)	payment_status character varying (20)	transaction_id character varying (255)
1	41385	CONFIRMED	COMPLETED	
2	41384	CONFIRMED	COMPLETED	
3	41383	CONFIRMED	COMPLETED	
4	41382	CONFIRMED	COMPLETED	
5	41381	CONFIRMED	COMPLETED	
6	41380	CONFIRMED	COMPLETED	
7	41379	CONFIRMED	COMPLETED	
8	41378	CONFIRMED	PENDING	
9	41377	CONFIRMED	PENDING	
10	41376	CONFIRMED	PENDING	
11	41375	CONFIRMED	PENDING	
12	41374	CONFIRMED	PENDING	
13	41373	CONFIRMED	PENDING	
14	41372	CONFIRMED	PENDING	
15	41371	CONFIRMED	PENDING	

After: `POST /api/orders/<id>/process-payment/` The `process_payment()` service ensures that a payment is processed only once by wrapping the operation inside `transaction.atomic()` and locking both the Order and related Payment records using `select_for_update()`.

This prevents multiple concurrent requests from modifying the same payment data at the same time. Before completing the payment, the service validates the order state and rejects cancelled orders. It also checks whether the payment is already marked as COMPLETED.

If a duplicate payment attempt is detected, the system rejects it with a clear response instead of processing the payment again.

Safe Result:

Only the first valid payment request succeeds. Any repeated or concurrent payment attempt is safely rejected, preventing double payment processing and keeping the order/payment state consistent.

this from ui of locust

 LOCUST

Host

http://nginx:80

Status

STOPPED

RPS

4.74

Failures

11%

NEW

RESET



STATISTICS

CHARTS

FAILURES

EXCEPTIONS

CURRENT RATIO

DOWNLOAD DATA

LOGS



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/api/users/login/ [race1-after]	47	0	36000	44000	50000	32697.76	1940	49595	52	2	0
POST	Req1 AFTER Case 1 AFTER POST /api/orders/checkout/ (locked checkout)	37	0	1200	26000	27000	3187.65	75	26778	373.08	1.8	0
POST	Req1 AFTER Case 2 POST /api/orders/checkout-wallet-async/ (wallet double-spend protected)	24	0	270	4800	8600	1058.13	64	8598	421.08	1	0
POST	Req1 AFTER Case 3 double payment POST /api/orders/{id}/process-payment/	28	0	240	4700	35000	1762.95	47	35310	79.5	1.1	0

result from our terminal

```

CASE 3 – Double Payment Processing
  BEFORE endpoint : POST /api/orders/<id>/process-payment-unsafe/ twice
  AFTER  endpoint : POST /api/orders/<id>/process-payment/ twice after safe order
Scenario : same customer tries to pay the same order twice – order+payment rows locked
First payment ok (200)      : 14
Duplicate blocked (400)     : 14
✓ PROTECTED – double payment prevented by pessimistic lock

```

3.7 Case 4: Cancel Order While Payment Is Processing

This situation occurs when a cancellation request is submitted at the same time that payment for the same order is being processed.

Before the solution: Both the cancel and process payment requests attempt to modify the same order concurrently.

Potential outcome: The order may reach an illogical state, such as being cancelled after its payment has already been completed.

Before: `POST /api/orders/<id>/cancel-unsafe/`

The unsafe cancellation endpoint does not lock the order row and does not enforce the order state machine. As a result, payment and cancellation can update the same order independently, producing an invalid state where a completed payment is followed by order cancellation.

The high failure rate in Locust confirms this race condition and proves that row-level locking with state validation is required to prevent paid orders from being cancelled incorrectly.

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/api/users/login/ [race1-before]	80	0	27000	36000	37000	26431.5	5766	37496	52	0	0
POST	Req1 BEFORE Case 1 BEFORE POST /api/orders/race-demo/ (unsafe checkout)	2722	1374	360	840	3200	485.11	124	23070	154.32	51	27.9
POST	Req1 BEFORE Case 2 POST /api/orders/blocking-wallet-checkout/ (wallet double-spend)	880	809	190	3300	3600	479.38	27	6437	162.59	19.3	18.8
POST	Req1 BEFORE Case 3 double payment POST /api/orders/{id}/process-payment-unsafe/	1779	888	310	590	1100	344.28	128	4851	214.5	30.8	15.3
POST	Req1 BEFORE Case 4 cancel while payment processing POST /api/orders/{id}/cancel-unsafe/	442	441	290	510	890	314.19	129	1709	186	8.3	8.3

The error `409 Conflict: paid order was cancelled` shows that payment was completed, but the unsafe cancellation endpoint still cancelled the order. This confirms that concurrent operations can create inconsistent data before applying the safe solution.

```
POST Req1 BEFORE Case 4 cancel while payment processing | CatchResponseError(409 Conflict: paid order was cancelled order=43248)
POST /api/orders/{id}/cancel-unsafe/
```

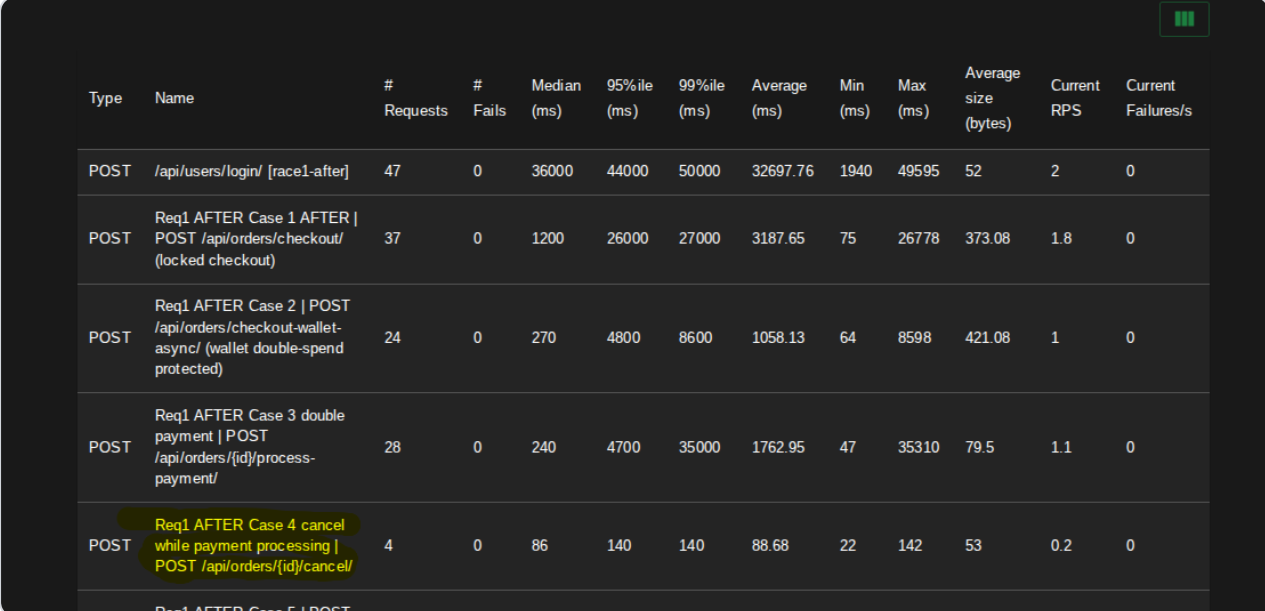
this terminal when finish test:

The terminal counts blocked requests, while Locust counts unexpected failures. In this case, HTTP 400 is expected because the protected endpoint should reject invalid cancellation attempts. Therefore, the terminal shows blocked requests, but Locust reports zero failures because the system behaved correctly.

```
CASE 4 – Cancel Order While Payment Is Processing
BEFORE endpoint : POST /api/orders/<id>/cancel-unsafe/ after payment attempt
AFTER endpoint  : POST /api/orders/<id>/cancel/ after safe order/payment attempt
Scenario : cancel races with payment – order row lock and state machine enforced
Cancel ok (200) : 1
Blocked – wrong state (400) : 441
```

This terminal output shows Requirement 1 - Case 4 after the solution. The protected `/api/orders/<id>/cancel/` endpoint uses row-level locking and state validation.

Although the terminal shows `Blocked - wrong state (400)`, Locust reports zero failures because HTTP 400 is the expected safe response in this scenario. The protected endpoint correctly rejects invalid cancellation attempts, so Locust marks these responses as successful behavior rather than test failures.



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/api/users/login/ [race1-after]	47	0	36000	44000	50000	32697.76	1940	49595	52	2	0
POST	Req1 AFTER Case 1 AFTER POST /api/orders/checkout/ (locked checkout)	37	0	1200	26000	27000	3187.65	75	26778	373.08	1.8	0
POST	Req1 AFTER Case 2 POST /api/orders/checkout-wallet-async/ (wallet double-spend protected)	24	0	270	4800	8600	1058.13	64	8598	421.08	1	0
POST	Req1 AFTER Case 3 double payment POST /api/orders/{id}/process-payment/	28	0	240	4700	35000	1762.95	47	35310	79.5	1.1	0
POST	Req1 AFTER Case 4 cancel while payment processing POST /api/orders/{id}/cancel/	4	0	86	140	140	88.68	22	142	53	0.2	0
POST	Req1 AFTER Case 5 POST											

Only 1 cancellation succeeded, while 441 requests were blocked with HTTP 400, confirming that paid or processing orders can no longer be cancelled incorrectly during concurrent payment activity.

The database snapshot joins the latest orders with their payment records to detect inconsistent state transitions during concurrent cancellation and payment processing. It helps verify whether an order reached an invalid combination, such as completed payment with incorrect order cancellation.

Query

Query History

Scratch Pad

8FROM orders_order o

9JOIN orders_payment p ON p.order_id = o.id

10ORDER BY o.id DESC

11LIMIT 20;

12

Data Output

Messages

Notifications

<

After: `POST /api/orders/<id>/cancel/`

The safe service `cancel_order()` locks the order row with `select_for_update()` and only permits cancellation while the order is `PENDING` or `CONFIRMED`. Once payment processing changes the order to `PROCESSING`, cancellation is rejected.

The solution uses **pessimistic locking** with `transaction.atomic()` and `select_for_update()` to lock the order row during both payment processing and cancellation. It also applies state validation to ensure that each order status transition is valid.

Safe outcome: If payment is completed first, cancellation is blocked. If cancellation happens first, payment processing is blocked. Therefore, an order can no longer become both `CANCELLED` and `PAID` at the same time.

The HTTP 400 responses are expected after the solution. They indicate that the protected endpoint correctly rejected cancellation requests when the order state did not allow cancellation, preventing a paid order from being cancelled.

CASE 4 – Cancel Order While Payment Is Processing

BEFORE endpoint : `POST /api/orders/<id>/cancel-unsafe/` after payment attempt

AFTER endpoint : `POST /api/orders/<id>/cancel/` after safe order/payment attempt

Scenario : cancel races with payment – order row lock and state machine enforced

Cancel ok (200) : 0

Blocked – wrong state (400) : 4

no paid order is shown as cancelled, confirming that the protected endpoint prevents invalid order/payment states.

The screenshot shows a database query interface with a query editor and a results table. The query is as follows:

```
5  p.method,  
6  p.transaction_id,  
7  o.updated_at  
8  FROM orders_order o  
9  JOIN orders_payment p ON p.order_id = o.id  
10 ORDER BY o.id DESC  
11 LIMIT 20;
```

The results table displays 20 rows of data. The columns are: order_id, order_status, payment_status, method, transaction_id, and updated_at. The data shows a mix of CONFIRMED, PENDING, and PROCESSING orders with various payment methods and transaction IDs.

order_id	order_status	payment_status	method	transaction_id	updated_at
47016	CONFIRMED	PENDING	CREDIT_CARD		2026-05-18 19:25:07.203074+00
47015	CONFIRMED	PENDING	CREDIT_CARD		2026-05-18 19:25:06.945684+00
47014	CONFIRMED	PENDING	CREDIT_CARD		2026-05-18 19:25:06.776249+00
47013	CONFIRMED	PENDING	CREDIT_CARD		2026-05-18 19:25:06.298114+00
47012	CONFIRMED	PENDING	CREDIT_CARD		2026-05-18 19:25:06.141041+00
47011	PROCESSING	COMPLETED	credit_card	req1-47011-1	2026-05-18 19:25:06.23251+00
47010	CONFIRMED	PENDING	CREDIT_CARD		2026-05-18 19:25:05.619919+00
47009	PROCESSING	COMPLETED	credit_card	req1-47009-1	2026-05-18 19:25:05.737196+00
47008	CONFIRMED	PENDING	CREDIT_CARD		2026-05-18 19:25:05.143167+00
47007	CONFIRMED	COMPLETED	WALLET		2026-05-18 19:25:04.296269+00
47006	CONFIRMED	PENDING	CREDIT_CARD		2026-05-18 19:25:04.365157+00
47005	CONFIRMED	PENDING	CREDIT_CARD		2026-05-18 19:25:04.236089+00
47004	CONFIRMED	PENDING	CREDIT_CARD		2026-05-18 19:25:03.972188+00
47003	PROCESSING	COMPLETED	credit_card	req1-47003-1	2026-05-18 19:25:04.396002+00
47002	PROCESSING	COMPLETED	credit_card	req1-47002-1	2026-05-18 19:25:03.518861+00
47001	PROCESSING	COMPLETED	credit_card	req1-47001-1	2026-05-18 19:25:03.853994+00
47000	PROCESSING	COMPLETED	credit_card	req1-47000-1	2026-05-18 19:25:03.419098+00

Total rows: 20 Query complete 00:00:00.192

3.8 Case 5: Product Reservation / Over-Reservation

This situation occurs when multiple users try to reserve the same low-stock product at the same time.

Before the solution: The unsafe reservation endpoint creates product reservations without locking the product row or checking the latest reserved quantity atomically.

Potential outcome: The total active reserved quantity can become greater than the actual available stock, which means the system has over-reserved the product.

Before: `POST /api/products/<id>/reserve-unsafe/`

The unsafe endpoint calculates active reservations without locking, sleeps for 100 ms, then creates a new `OrderLock`. Concurrent users can all see the same old reserved quantity and create reservations that exceed product stock.

Locust statistics for Requirement 1 - Case 5 BEFORE. The unsafe reservation endpoint `/api/products/{id}/reserve-unsafe/` received 357 requests and all of them failed, showing that the test successfully exposed the over-reservation race condition.

LOCUST

Host

http://nginx:80

Status

STOPPED

RPS

23.85

Failures

67%

NEW

RESET

STATISTICS

CHARTS

FAILURES

EXCEPTIONS

CURRENT RATIO

DOWNLOAD DATA

LOGS

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/api/users/login/ [race1-before]	80	60	60000	60000	60000	57046.18	19714	60202	138.25	0	0
POST	Req1 BEFORE Case 1 BEFORE POST /api/orders/race-demo/ (unsafe checkout)	973	630	170	750	1100	278.2	2	37943	154.88	32.8	20.2
POST	Req1 BEFORE Case 2 POST /api/orders/blocking-wallet-checkout/ (wallet double-spend)	201	188	81	3300	7500	520.77	18	9573	155.48	6.1	5.9
POST	Req1 BEFORE Case 3 double payment POST /api/orders/{id}/process-payment-unsafe/	446	222	210	730	1500	365.19	116	29719	214.5	15.6	7.9
POST	Req1 BEFORE Case 4 cancel while payment processing POST /api/orders/{id}/cancel-unsafe/	117	104	180	540	740	281.93	117	6130	185.78	4.7	4.2
POST	Req1 BEFORE Case 5 POST /api/products/{id}/reserve-unsafe/ (over-reservation)	357	357	170	690	1000	207.35	2	2605	175.89	12	12

The HTTP 400 responses are expected after the solution. They indicate that the protected endpoint correctly rejected reservation requests when stock was insufficient, instead of allowing over-reservation.

CASE 5 – Product Reservation / Over-Reservation

BEFORE endpoint : POST `/api/products/<id>/reserve-unsafe/` (no product row lock)

AFTER endpoint : POST `/api/products/<id>/reserve/` (SELECT FOR UPDATE)

Scenario : concurrent reservations for same product – unsafe over-reserves, safe blocks

Reservation ok (201) : 3

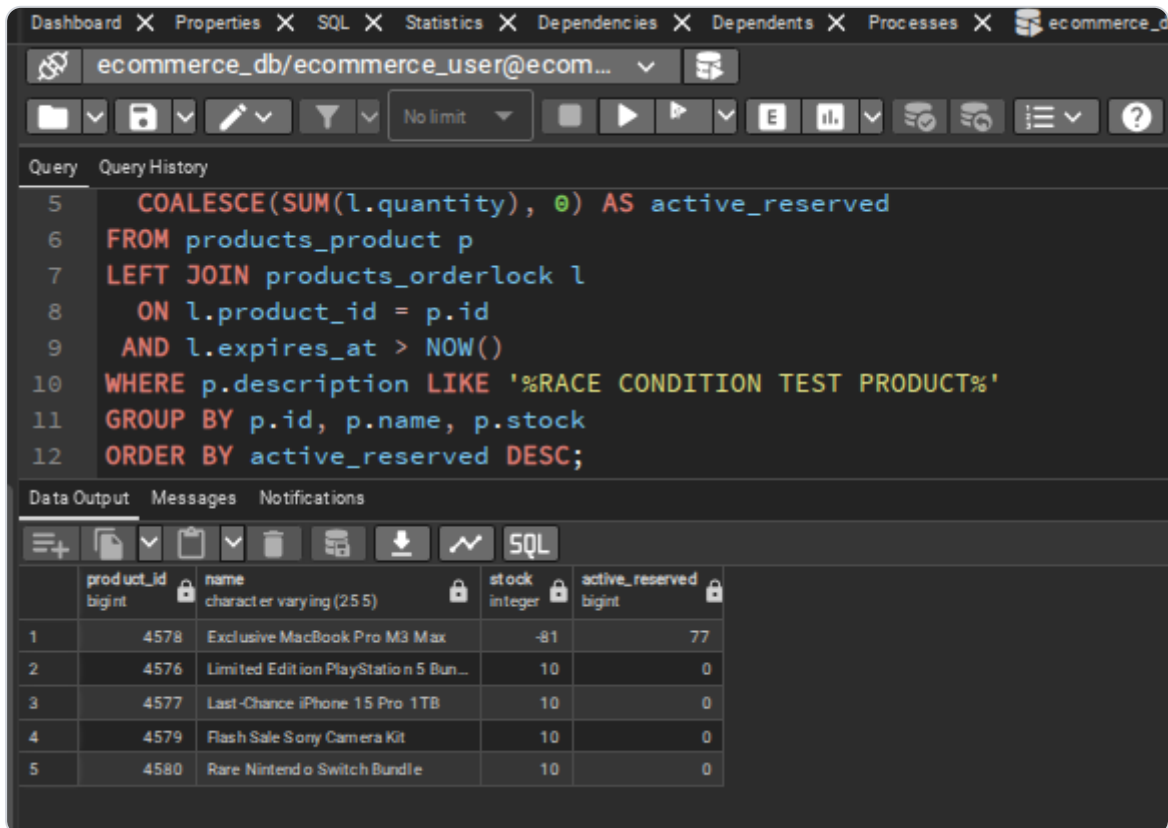
Blocked – low stock (400) : 23

The failure message `OVER-RESERVED: reserved=114 stock=149` shows that concurrent reservation requests caused the active reserved quantity to exceed the expected safe reservation limit. This confirms that the unsafe endpoint allowed inconsistent reservation

behavior before row-level locking.

```
Req1 BEFORE Case 5 | POST
/api/products/{id}/reserve-unsafe/ (over-
reservation) CatchResponseError(OVER-RESERVED: reserved=114 stock=-149')
```

The database shows an invalid reservation state before the solution: one low-stock product has negative stock and active reservations at the same time. This confirms that concurrent unsafe reservation requests can corrupt inventory without row-level locking.



The screenshot shows a database management tool interface. The top bar includes tabs for Dashboard, Properties, SQL, Statistics, Dependencies, Dependents, Processes, and ecommerce_d. The main area displays a SQL query in the 'Query' tab, which is a complex query involving a COALESCE function, a LEFT JOIN, and a WHERE clause. Below the query, the 'Data Output' tab shows a table with 5 rows and 5 columns: product_id, name, stock, active_reserved, and another column. The table contains data for various products, including MacBook Pro, PlayStation 5, iPhone 15 Pro, Sony Camera Kit, and Nintendo Switch Bundle.

	product_id bigint	name character varying (255)	stock integer	active_reserved bigint
1	4578	Exclusive MacBook Pro M3 Max	-81	77
2	4576	Limited Edition PlayStation 5 Bun...	10	0
3	4577	Last-Chance iPhone 15 Pro 1TB	10	0
4	4579	Flash Sale Sony Camera Kit	10	0
5	4580	Rare Nintendo Switch Bundle	10	0

and this from terminal

```
CASE 5 – Product Reservation / Over-Reservation
BEFORE endpoint : POST /api/products/<id>/reserve-unsafe/ (no product row lock)
AFTER  endpoint : POST /api/products/<id>/reserve/          (SELECT FOR UPDATE)
Scenario : concurrent reservations for same product – unsafe over-reserves, safe blocks
Reservation ok (201)           : 5
Over-reserved / blocked count : 294
```

After: `POST /api/products/<id>/reserve/`

The safe endpoint calls `create_order_lock()`, which wraps the reservation in a transaction and locks the product row with `select_for_update()`. The request only creates the reservation if enough stock is available.

The solution uses **pessimistic locking** with `transaction.atomic()` and `select_for_update()` to lock the product row during the reservation process. The system locks the product first, checks the available quantity, and creates the reservation only if enough stock is available.

Safe outcome: Reservations are created only within the available stock limit, and any extra reservation request is rejected.

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/api/users/login/ [race1-after]	47	0	36000	44000	50000	32697.76	1940	49595	52	2	0
POST	Req1 AFTER Case 1 AFTER POST /api/orders/checkout/ (locked checkout)	37	0	1200	26000	27000	3187.65	75	26778	373.08	1.8	0
POST	Req1 AFTER Case 2 POST /api/orders/checkout-wallet-async/ (wallet double-spend protected)	24	0	270	4800	8600	1058.13	64	8598	421.08	1	0
POST	Req1 AFTER Case 3 double payment POST /api/orders/{id}/process-payment/	28	0	240	4700	35000	1762.95	47	35310	79.5	1.1	0
POST	Req1 AFTER Case 4 cancel while payment processing POST /api/orders/{id}/cancel/	4	0	86	140	140	88.68	22	142	53	0.2	0
POST	Req1 AFTER Case 5 POST /api/products/{id}/reserve/ (over-reservation protected)	26	0	260	4200	4700	692.6	22	4734	50.96	1.6	0

The HTTP 400 responses are expected after the solution. They indicate that the protected endpoint correctly rejected reservation requests when stock was insufficient, instead of allowing over-reservation.

CASE 5 – Product Reservation / Over-Reservation

BEFORE endpoint : POST /api/products/<id>/reserve-unsafe/ (no product row lock)

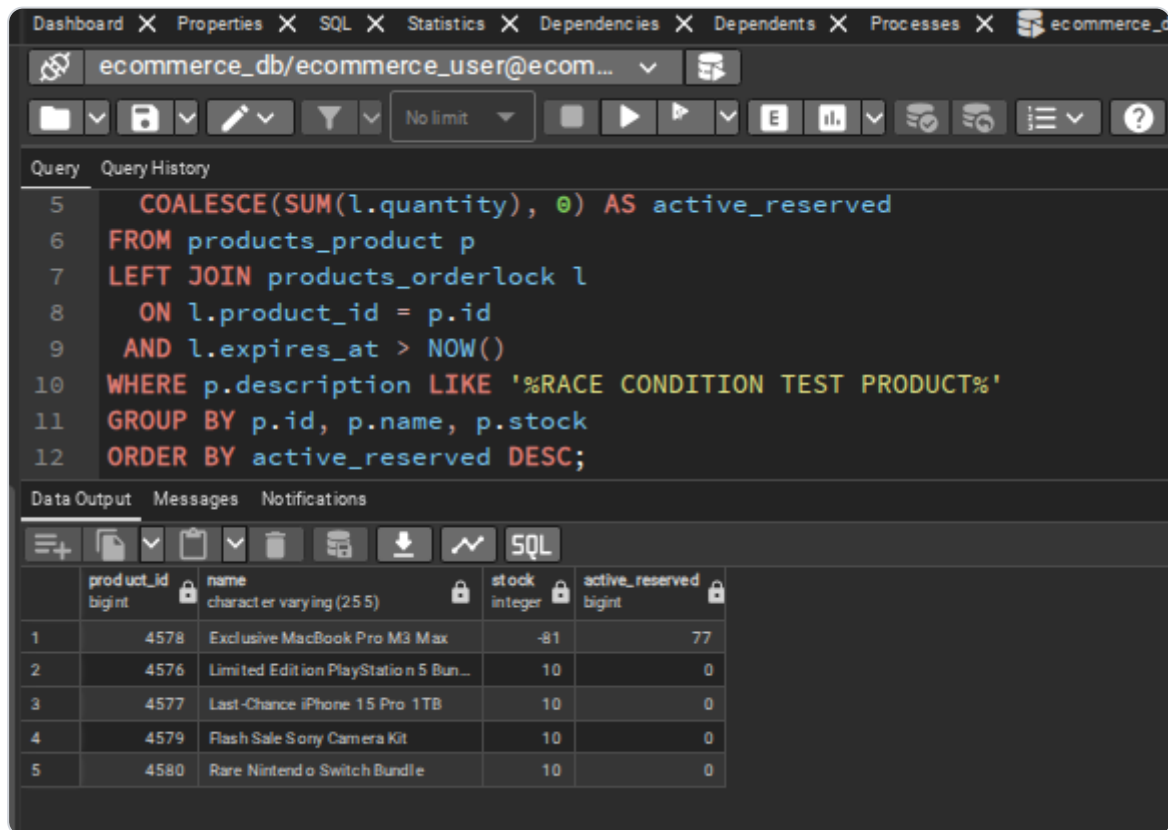
AFTER endpoint : POST /api/products/<id>/reserve/ (SELECT FOR UPDATE)

Scenario : concurrent reservations for same product – unsafe over-reserves, safe blocks

Reservation ok (201) : 3

Blocked – low stock (400) : 23

db:After the solution, all race-condition products remain in a **SAFE** reservation state. The active reserved quantity is within the available stock, confirming that pessimistic locking prevents over-reservation.



The screenshot shows a database management interface with a query editor and a results table. The query is as follows:

```
5 COALESCE(SUM(l.quantity), 0) AS active_reserved
6 FROM products_product p
7 LEFT JOIN products_orderlock l
8 ON l.product_id = p.id
9 AND l.expires_at > NOW()
10 WHERE p.description LIKE '%RACE CONDITION TEST PRODUCT%'
11 GROUP BY p.id, p.name, p.stock
12 ORDER BY active_reserved DESC;
```

The results table displays the following data:

	product_id bigint	name character varying (255)	stock integer	active_reserved bigint
1	4578	Exclusive MacBook Pro M3 Max	-81	77
2	4576	Limited Edition PlayStation 5 Bun...	10	0
3	4577	Last-Chance iPhone 15 Pro 1TB	10	0
4	4579	Flash Sale Sony Camera Kit	10	0
5	4580	Rare Nintendo Switch Bundle	10	0

4. Requirement 2 - Resource Management and Capacity Control

4.1 Problem

The backend must control parallel work so it does not collapse under load or become slow because of under-utilization. Concurrency exists at several layers: HTTP request handling, database connections, background workers, and retry behavior.

4.2 Implemented Controls

Layer	Control
HTTP server	Gunicorn <code>gthread</code> worker class
Per app container	<code>--workers=8 --threads=3</code>
Total app capacity	<code>5 containers x 8 workers x 3 threads = 120 request handlers</code>
Database connection pressure	PgBouncer transaction pooling
PostgreSQL headroom	<code>max_connections=200</code>
Django connection reuse	<code>CONN_MAX_AGE=60</code>
Celery capacity	Queue-specific workers and fixed concurrency
Retry pressure	Optimistic stock retry uses short backoff

4.3 Current Gunicorn Configuration

Each app container starts with:

```
gunicorn --worker-class=gthread --workers=8 --threads=3 \
  --access-logfile=- --error-logfile=- \
  --bind=0.0.0.0:8000 config.wsgi:application
```

Capacity calculation:

```
Each app container: 8 workers x 3 threads = 24 request handlers
All app containers: 5 x 24 = 120 request handlers
```

4.4 Database Connection Control

Django and Celery connect to PgBouncer at:

```
DB_HOST=pgbouncer
DB_PORT=5432
```

PgBouncer then connects to PostgreSQL at `db:5432` . This protects PostgreSQL from one direct connection per request thread or Celery task.

Current PgBouncer settings:

```
P00L_MODE=transaction
MAX_CLIENT_CONN=500
DEFAULT_P00L_SIZE=30
RESERVE_P00L_SIZE=10
```

4.5 Celery Capacity Control

Worker	Queue	Concurrency	Purpose
<code>celery_worker</code>	<code>celery</code>	2	General tasks
<code>celery_email_worker</code>	<code>emails</code>	4	Email and notification tasks
<code>celery_batch_worker</code>	<code>batch</code>	1	Heavy database batch jobs

The batch worker intentionally uses low concurrency because batch aggregation is database-heavy.

4.6 Validation Commands

Run capacity test:

Obtain token using admin credentials:

```
curl -X POST http://localhost/api/users/login/ \
  -H "Content-Type: application/json" \
  -d '{"email":"admin@demo.com","password":"admin123"}'
```

then:

```
docker compose run --rm \
  -e LOCUST_MODE=req2 \
  -e ADMIN_TOKEN="<admin-token>" \
  locust -f locustfile.py --host http://nginx:80 \
  --users 500 --spawn-rate 50 --headless --run-time 3m
```

Monitor containers:

```
docker stats ecommerce_app1 ecommerce_app2 ecommerce_app3 ecommerce_app4
ecommerce_app5 \
  ecommerce_pggbouncer ecommerce_db ecommerce_nginx ecommerce_redis
```

CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
6e3ad62188df	ecommerce_app1	51.74%	468MiB / 14.83GiB	3.08%	3.09MB / 3.57MB	24.6kB / 0B	32
161206695adc	ecommerce_app2	52.46%	451.9MiB / 14.83GiB	2.98%	3.08MB / 3.58MB	0B / 0B	27
e4b69b0f0cc1	ecommerce_app3	54.46%	467.9MiB / 14.83GiB	3.08%	3.06MB / 3.55MB	0B / 0B	30
1c83df10a54c	ecommerce_app4	55.67%	467.6MiB / 14.83GiB	3.08%	3.06MB / 3.56MB	393kB / 0B	28
87b7cf93e36f	ecommerce_app5	55.95%	468.3MiB / 14.83GiB	3.08%	3.05MB / 3.54MB	131kB / 0B	29
42709b451826	ecommerce_pggbouncer	5.15%	5.926MiB / 14.83GiB	0.04%	15.4MB / 16.3MB	184kB / 8.19kB	1
9d2076f0b10a	ecommerce_db	13.79%	53.41MiB / 14.83GiB	0.35%	4.11MB / 5.03MB	13.4MB / 291kB	16
4fd155e7cc23	ecommerce_nginx	4.56%	27.07MiB / 14.83GiB	0.18%	10MB / 10.3MB	1.89MB / 4.1kB	25
5ccf4df61b8e	ecommerce_redis	0.24%	24.29MiB / 14.83GiB	0.16%	189kB / 243kB	57.9MB / 3.64MB	6

Monitor active database connections:

```
docker exec ecommerce_db psql -U ecommerce_user -d ecommerce_db \
  -c "SELECT count(*), state FROM pg_stat_activity WHERE
  datname='ecommerce_db' GROUP BY state ORDER BY state;"
```

```
count | state
-----+-----
      1 | active
     10 | idle
(2 rows)
```

5. Requirement 3 - Asynchronous Queues

5.1 Problem

Some work should not block the HTTP response. Confirmation emails, cancellation emails, wallet payment simulation, expired-lock cleanup, and batch jobs are better handled outside the request path.

5.2 Before: Synchronous Work

The demo endpoint `POST /api/orders/checkout-sync/` creates an order and then simulates email sending with `sleep(2)` before returning HTTP 201.

The wallet sync endpoint `POST /api/orders/checkout-wallet-sync/` performs the 3-second payment gateway simulation inside the HTTP request. checkout → create order → wait email being sent inside request → response These are useful comparison points because the user waits for the slow work.

5.3 After: Celery Queues

The real checkout endpoint `POST /api/orders/checkout/` creates the order, queues `send_order_confirmation_email.delay(order.id)`, and immediately returns HTTP 201.

The wallet async endpoint `POST /api/orders/checkout-wallet-async/` queues `process_wallet_payment_async.delay(user.id)` and immediately returns HTTP 202.

Task routing is separated in `config/settings.py`:

Task Type	Queue
Order confirmation email	emails
Order cancellation email	emails
Daily sales batch	batch
Abandoned cart cleanup	batch
Expired product locks cleanup	celery
Product cache invalidation	celery

we speratead the queues based on their purpose, so we dont end up with one heavily stacked queue.

5.4 Validation Commands

Synchronous email comparison:


```
docker logs -f ecommerce_celery_email_worker:
```

the worker is empty because the emails are being sent inside the HTTP request lifetime and not offloaded to Celery.

```
- *** --- * ---
- ** ----- [config]
- ** ----- .> app:          config:0x7f03c5a4b5d0
- ** ----- .> transport:   redis://redis:6379/0
- ** ----- .> results:    redis://redis:6379/1
- *** --- * ---
- ** ----- .> concurrency: 4 (prefork)
- ***** --- .> task events: OFF (enable -E to monitor tasks in this worker)
- ***** ---
- ----- [queues]
- .> emails          exchange=celery(direct) key=celery

[tasks]
. apps.cart.tasks.cleanup_abandoned_carts
. apps.core.tasks.cleanup_abandoned_carts
. apps.core.tasks.run_daily_sales_batch_naive_task
. apps.core.tasks.run_daily_sales_batch_task
. apps.orders.tasks.send_order_cancelled_email
. apps.orders.tasks.send_order_confirmation_email
. apps.orders.tasks.update_order_status_task
. apps.products.tasks.alert_low_stock
. apps.products.tasks.invalidate_product_cache
. apps.products.tasks.release_expired_locks_task
. apps.users.tasks.send_password_reset_email
. apps.users.tasks.send_welcome_email
. config.celery.debug_task

[2026-05-13 11:19:38,914: INFO/MainProcess] Connected to redis://redis:6379/0
[2026-05-13 11:19:38,918: INFO/MainProcess] mingle: searching for neighbors
[2026-05-13 11:19:39,929: INFO/MainProcess] mingle: all alone
[2026-05-13 11:19:39,942: INFO/MainProcess] email_worker@31f812c6db55 ready.
```

```
docker compose run --rm \
  -e LOCUST_MODE=req3_sync \
  locust -f locustfile.py --host http://nginx:80 \
  --users 5 --spawn-rate 1 --headless --run-time 30s
```

POST	/api/users/login/ [sync-before]	5	0(0.00%)	1187	615	2398	960	0.00	0.00
POST	POST /api/orders/checkout-sync/ [REQ3 BEFORE - SLOW]	20	0(0.00%)	2562	2019	4030	2019		
0.90	0.00								
----- ----- ----- ----- ----- ----- ----- ----- ----- -----									
	Aggregated	71	0(0.00%)	1048	4	4030	720	2.50	0.00
Type	Name	# reqs	# fails	Avg	Min	Max	Med	req/s	failures/s
----- ----- ----- ----- ----- ----- ----- ----- ----- -----									
POST	/api/cart/add/ [sync-demo]	25	0(0.00%)	16	9	27	16	0.80	0.00
DELETE	/api/cart/clear/ [sync]	25	0(0.00%)	737	4	1640	730	0.80	0.00
POST	/api/users/login/ [sync-before]	5	0(0.00%)	1187	615	2398	960	0.00	0.00
POST	POST /api/orders/checkout-sync/ [REQ3 BEFORE - SLOW]	22	0(0.00%)	2580	2019	4030	2019		
0.90	0.00								
----- ----- ----- ----- ----- ----- ----- ----- ----- -----									
	Aggregated	77	0(0.00%)	1059	4	4030	720	2.50	0.00
Type	Name	# reqs	# fails	Avg	Min	Max	Med	req/s	failures/s
----- ----- ----- ----- ----- ----- ----- ----- ----- -----									
POST	/api/cart/add/ [sync-demo]	27	0(0.00%)	16	9	27	16	0.80	0.00
DELETE	/api/cart/clear/ [sync]	27	0(0.00%)	722	4	1640	730	0.80	0.00
POST	/api/users/login/ [sync-before]	5	0(0.00%)	1187	615	2398	960	0.00	0.00
POST	POST /api/orders/checkout-sync/ [REQ3 BEFORE - SLOW]	24	0(0.00%)	2594	2019	4030	2019		
0.90	0.00								
----- ----- ----- ----- ----- ----- ----- ----- ----- -----									
	Aggregated	83	0(0.00%)	1062	4	4030	720	2.50	0.00
[2026-05-13 11:49:17,369] 55e5958f8fa3/INFO/locust.main: --run-time limit reached, shutting down									

Mode: req3_sync									
REQ 1 - Race Condition Safety									
HOT product checkouts succeeded : 0									
Stock-outs blocked by locking : 0									
REQ 2 - Resource Management & Capacity Control									
Total requests : 0									
Errors : 0									
Evidence to screenshot:									
- Locust response time									
- docker stats									
- pg_stat_activity									
- active_queues for Celery workers									
- Docker Compose workers/concurrency									
- Exponential backoff code in services.py									
✔ No resource exhaustion detected									
REQ 3 - Async Queues (BEFORE vs AFTER)									
BEFORE (sync email): avg=2606ms samples=26									
REQ 4 - Batch Processing									
REQ 6 - Distributed Caching									
----- ----- ----- ----- ----- ----- ----- ----- ----- -----									
[2026-05-13 11:49:17,417] 55e5958f8fa3/INFO/locust.main: Shutting down (exit code 0)									
Type	Name	# reqs	# fails	Avg	Min	Max	Med		
----- ----- ----- ----- ----- ----- ----- ----- ----- -----									
POST	/api/cart/add/ [sync-demo]	29	0(0.00%)	16	9	27	16		

POST /api/orders/checkout-sync/ [REQ3 BEFORE — SLOW]

Avg ≈ 2100ms

Min $\approx$ 2017ms

Max $\approx$ 2300ms

Fails = 0

when watching the worker:

```
- *** --- * ---
- ** ----- [config]
- ** ----- .> app:      config:0x7f03c5a4b5d0
- ** ----- .> transport: redis://redis:6379/0
- ** ----- .> results:  redis://redis:6379/1
- *** --- * ---
- ** ----- .> concurrency: 4 (prefork)
- ***** --- .> task events: OFF (enable -E to monitor tasks in this worker)
- ***** ---
- ----- [queues]
- .> emails      exchange=celery(direct) key=celery

[tasks]
. apps.cart.tasks.cleanup_abandoned_carts
. apps.core.tasks.cleanup_abandoned_carts
. apps.core.tasks.run_daily_sales_batch_naive_task
. apps.core.tasks.run_daily_sales_batch_task
. apps.orders.tasks.send_order_cancelled_email
. apps.orders.tasks.send_order_confirmation_email
. apps.orders.tasks.update_order_status_task
. apps.products.tasks.alert_low_stock
. apps.products.tasks.invalidate_product_cache
. apps.products.tasks.release_expired_locks_task
. apps.users.tasks.send_password_reset_email
. apps.users.tasks.send_welcome_email
. config.celery.debug_task

[2026-05-13 11:19:38,914: INFO/MainProcess] Connected to redis://redis:6379/0
[2026-05-13 11:19:38,918: INFO/MainProcess] mingle: searching for neighbors
[2026-05-13 11:19:39,929: INFO/MainProcess] mingle: all alone
[2026-05-13 11:19:39,942: INFO/MainProcess] email_worker@31f812c6db55 ready.
```

Async email comparison:

```
docker compose run --rm \
  -e LOCUST_MODE=req3_async \
  locust -f locustfile.py --host http://nginx:80 \
  --users 5 --spawn-rate 1 --headless --run-time 30s
```

Aggregated		3	1(33.33%)	3655	15	5692	5300	0.12	0.12
Type	Name	# reqs	# fails	Avg	Min	Max	Med	req/s	failures/s
POST	/api/users/login/ [async-after]	3	1(33.33%)	3655	15	5692	5300	0.12	0.12
Aggregated		3	1(33.33%)	3655	15	5692	5300	0.12	0.12
Type	Name	# reqs	# fails	Avg	Min	Max	Med	req/s	failures/s
POST	/api/cart/add/ [async-demo]	4	0(0.00%)	95	68	119	89	0.00	0.00
DELETE	/api/cart/clear/ [async]	4	0(0.00%)	2556	50	5003	740	0.00	0.00
POST	/api/users/login/ [async-after]	5	1(20.00%)	5323	15	7971	5700	0.30	0.10
Aggregated		13	1(7.69%)	2863	15	7971	740	0.30	0.10
Type	Name	# reqs	# fails	Avg	Min	Max	Med	req/s	failures/s
POST	/api/cart/add/ [async-demo]	7	0(0.00%)	74	32	119	69	0.00	0.00
DELETE	/api/cart/clear/ [async]	9	0(0.00%)	1160	16	5003	50	0.00	0.00
POST	/api/users/login/ [async-after]	5	1(20.00%)	5323	15	7971	5700	0.40	0.10
POST	POST /api/orders/checkout/ [REQ3 AFTER - FAST]	6	0(0.00%)	374	83	637	330	0.00	0.00
Aggregated		27	1(3.70%)	1474	15	7971	100	0.40	0.10
Type	Name	# reqs	# fails	Avg	Min	Max	Med	req/s	failures/s
POST	/api/cart/add/ [async-demo]	17	0(0.00%)	80	32	196	67	0.50	0.00
DELETE	/api/cart/clear/ [async]	17	0(0.00%)	636	16	5003	49	0.50	0.00
POST	/api/users/login/ [async-after]	6	1(16.67%)	5993	15	9346	5700	0.40	0.00
POST	POST /api/orders/checkout/ [REQ3 AFTER - FAST]	15	0(0.00%)	249	69	637	180	0.40	0.00

REQUIREMENTS TEST REPORT

Mode: req3_async

REQ 1 – Race Condition Safety

HOT product checkouts succeeded : 0
 Stock-outs blocked by locking : 0

REQ 2 – Resource Management & Capacity Control

Total requests : 0
 Errors : 0

Evidence to screenshot:

- Locust response time
- docker stats
- pg_stat_activity
- active_queues for Celery workers
- Docker Compose workers/concurrency
- Exponential backoff code in services.py

✓ No resource exhaustion detected

REQ 3 – Async Queues (BEFORE vs AFTER)

AFTER (async Celery): avg=163ms samples=59

REQ 4 – Batch Processing

REQ 6 – Distributed Caching

REQ 3 – Async Queues (BEFORE vs AFTER)

AFTER (async Celery): avg=163ms samples=59

Inspect queues:

```
docker compose exec celery_worker celery -A config inspect active_queues
docker compose exec celery_email_worker celery -A config inspect
active_queues
docker compose exec celery_batch_worker celery -A config inspect
active_queues
```

```
[2026-05-13 11:19:39,929: INFO/MainProcess] mingie: all alone
[2026-05-13 11:19:39,942: INFO/MainProcess] email_worker@31f812c6db55 ready.
[2026-05-13 12:23:51,892: INFO/MainProcess] Task apps.orders.tasks.send_order_confirmation_email[2b9e2d08-bc07-4281-8808-04b7fbae0a71] received
[2026-05-13 12:23:51,971: INFO/MainProcess] Task apps.orders.tasks.send_order_confirmation_email[e8d79bd7-0748-4807-a895-520288ea340b] received
[2026-05-13 12:23:52,133: INFO/ForkPoolWorker-4] [EMAIL] Simulating confirmation email for Order #26922 to locust_60@test.com (sleep 2s) ...
[2026-05-13 12:23:52,159: INFO/MainProcess] Task apps.orders.tasks.send_order_confirmation_email[1bb36504-1ddf-4ff2-a39f-da46f3397620] received
[2026-05-13 12:23:52,283: INFO/MainProcess] Task apps.orders.tasks.send_order_confirmation_email[0c7eefb1-02d2-4711-acfc-4e31bd546881] received
[2026-05-13 12:23:52,300: INFO/ForkPoolWorker-2] [EMAIL] Simulating confirmation email for Order #26923 to locust_17@test.com (sleep 2s) ...
[2026-05-13 12:23:52,955: INFO/MainProcess] Task apps.orders.tasks.send_order_confirmation_email[3559ca89-01df-4a13-8bec-e8f06a5434f6] received
[2026-05-13 12:23:53,512: INFO/MainProcess] Task apps.orders.tasks.send_order_confirmation_email[2992d75d-c339-41fc-aa88-099426a4e9de] received
[2026-05-13 12:23:53,714: INFO/MainProcess] Task apps.orders.tasks.send_order_confirmation_email[397b49e9-102e-40ae-9c90-7ba672e9b841] received
[2026-05-13 12:23:53,961: INFO/MainProcess] Task apps.orders.tasks.send_order_confirmation_email[47b48b76-ebd6-49dc-aa4a-8cbabebd70e7] received
[2026-05-13 12:23:54,052: INFO/MainProcess] Task apps.orders.tasks.send_order_confirmation_email[53d15954-26a9-42a0-a698-08cbaf82659c] received
[2026-05-13 12:23:54,134: INFO/ForkPoolWorker-4] [EMAIL] Confirmation email simulation complete for Order #26922
[2026-05-13 12:23:54,152: INFO/ForkPoolWorker-4] Task apps.orders.tasks.send_order_confirmation_email[2b9e2d08-bc07-4281-8808-04b7fbae0a71] succeeded
in 2.246826513999622s: None
[2026-05-13 12:23:54,302: INFO/ForkPoolWorker-2] [EMAIL] Confirmation email simulation complete for Order #26923
[2026-05-13 12:23:54,332: INFO/ForkPoolWorker-2] Task apps.orders.tasks.send_order_confirmation_email[e8d79bd7-0748-4807-a895-520288ea340b] succeeded
in 2.3369953300007182s: None
[2026-05-13 12:23:54,431: INFO/MainProcess] Task apps.orders.tasks.send_order_confirmation_email[deb54d3c-4c6b-40c5-98aa-90d5a4b90ccc] received
[2026-05-13 12:23:54,568: INFO/MainProcess] Task apps.orders.tasks.send_order_confirmation_email[91b9e62f-2323-477d-a76f-9efba8561246] received
[2026-05-13 12:23:54,585: INFO/MainProcess] Task apps.orders.tasks.send_order_confirmation_email[9296a7e3-730c-46b3-bcb3-4c7b00397d6a] received
[2026-05-13 12:23:54,706: INFO/MainProcess] Task apps.orders.tasks.send_order_confirmation_email[b14932bc-aeef-4dec-9dbb-dcfa211098c9] received
[2026-05-13 12:23:55,292: INFO/MainProcess] Task apps.orders.tasks.send_order_confirmation_email[f9f86097-a533-45a7-a5ed-5330f27ad448] received
[2026-05-13 12:23:55,554: INFO/MainProcess] Task apps.orders.tasks.send_order_confirmation_email[4e655fd8-3078-4b79-b040-a1bf465bb55e] received
[2026-05-13 12:23:55,831: INFO/MainProcess] Task apps.orders.tasks.send_order_confirmation_email[965f6973-7601-479c-8d09-d1f16842630b] received
[2026-05-13 12:23:55,882: INFO/MainProcess] Task apps.orders.tasks.send_order_confirmation_email[95662582-e792-4530-a669-74320e5ef562] received
[2026-05-13 12:23:56,129: INFO/MainProcess] Task apps.orders.tasks.send_order_confirmation_email[bedb15dd-125d-4b40-ba70-9bbcd9ffeb15] received
[2026-05-13 12:23:58,015: INFO/MainProcess] Task apps.orders.tasks.send_order_confirmation_email[58df7627-bc36-41dd-88b7-ffb79c56d820] received
```

Expected result:

HTTP response returns before the email/payment simulation finishes.
Celery worker logs show the background task processing later.

Comparison Table

Async queue	Sync	Measure
164ms	2594ms	Avg Response
69ms	2019ms	Min Response
637ms	4030ms	Max Response
works in Background	empty	Celery status
16x		speedup

6. Requirement 4 - Batch Processing

6.1 Problem

A daily sales aggregation can involve many orders. Loading all orders at once is simple but does not scale because memory usage grows with the number of orders.

6.2 Before: Naive Batch

The demo endpoint:

```
POST /api/core/trigger-batch-naive/
```

queues `run_daily_sales_batch_naive_task`. This task loads all matching `Order` objects into Python memory and processes them in one pass.

Expected log evidence:

```
[BATCH-NAIVE] Loaded ALL X order objects into memory at once -- NO  
CHUNKING
```

6.3 After: Chunked Batch

The optimized endpoint:

```
POST /api/core/trigger-batch/
```

queues `run_daily_sales_batch_task`. The task collects matching order IDs and processes them in fixed-size chunks. The production constant is:

```
CHUNK_SIZE = 50
```

For demonstrations, the endpoint accepts `chunk_size`, for example `{"chunk_size": 10}`, so chunks are visible even with fewer orders.

Expected log evidence:

```
[BATCH] Chunk 1/N processed ...  
[BATCH] Chunk 2/N processed ...  
[BATCH] Daily sales ... COMPLETE
```

6.4 Scheduling and Isolation

Celery Beat schedules the daily sales report at 1:00 AM:

```
apps.core.tasks.run_daily_sales_batch_task
```

The task is routed to the `batch` queue and consumed by `ecommerce_celery_batch_worker` with concurrency 1. This keeps heavy batch database work isolated from email notifications and normal background tasks.

6.5 Validation Commands

Naive batch:


```
docker compose run --rm \
-e LOCUST_MODE=req4_before \
-e ADMIN_TOKEN="<admin-token>" \
locust -f locustfile.py --host http://nginx:80 \
--users 1 --spawn-rate 1 --headless --run-time 30s
```

Chunked batch:

```
docker compose run --rm \
-e LOCUST_MODE=req4_after \
-e ADMIN_TOKEN="<admin-token>" \
locust -f locustfile.py --host http://nginx:80 \
--users 1 --spawn-rate 1 --headless --run-time 30s
```

Watch logs:

```
docker logs -f ecommerce_celery_batch_worker
```

```
[2026-05-18 11:48:50,815: INFO/MainProcess] Task apps.core.tasks.run_daily_sales_batch_task[bf45de5b-d3c0-4e4c-9473-1790a226eb7e] received
[2026-05-18 11:48:50,816: INFO/ForkPoolWorker-1] [BATCH] Starting daily sales aggregation for 2026-05-17 (window 2026-05-17 00:00:00+00:00 → 2026-05-18 00:00:00+00:00)
[2026-05-18 11:48:50,824: INFO/ForkPoolWorker-1] [BATCH] Chunk 1/348 processed: 10 orders, revenue=60645.50
[2026-05-18 11:48:50,824: INFO/ForkPoolWorker-1] [BATCH] Chunk 2/348 processed: 10 orders, revenue=42934.77
[2026-05-18 11:48:50,825: INFO/ForkPoolWorker-1] [BATCH] Chunk 3/348 processed: 10 orders, revenue=33963.43
[2026-05-18 11:48:50,825: INFO/ForkPoolWorker-1] [BATCH] Chunk 4/348 processed: 10 orders, revenue=7277.71
[2026-05-18 11:48:50,826: INFO/ForkPoolWorker-1] [BATCH] Chunk 5/348 processed: 10 orders, revenue=10303.72
[2026-05-18 11:48:50,826: INFO/ForkPoolWorker-1] [BATCH] Chunk 6/348 processed: 10 orders, revenue=12551.00
[2026-05-18 11:48:50,827: INFO/ForkPoolWorker-1] [BATCH] Chunk 7/348 processed: 10 orders, revenue=10995.29
[2026-05-18 11:48:50,827: INFO/ForkPoolWorker-1] [BATCH] Chunk 8/348 processed: 10 orders, revenue=9925.21
[2026-05-18 11:48:50,828: INFO/ForkPoolWorker-1] [BATCH] Chunk 9/348 processed: 10 orders, revenue=10015.93
[2026-05-18 11:48:50,828: INFO/ForkPoolWorker-1] [BATCH] Chunk 10/348 processed: 10 orders, revenue=15540.51
[2026-05-18 11:48:50,829: INFO/ForkPoolWorker-1] [BATCH] Chunk 11/348 processed: 10 orders, revenue=9493.07
[2026-05-18 11:48:50,829: INFO/ForkPoolWorker-1] [BATCH] Chunk 12/348 processed: 10 orders, revenue=10855.00
[2026-05-18 11:48:50,830: INFO/ForkPoolWorker-1] [BATCH] Chunk 13/348 processed: 10 orders, revenue=11599.50
[2026-05-18 11:48:50,830: INFO/ForkPoolWorker-1] [BATCH] Chunk 14/348 processed: 10 orders, revenue=10449.60
[2026-05-18 11:48:50,831: INFO/ForkPoolWorker-1] [BATCH] Chunk 15/348 processed: 10 orders, revenue=9305.07
[2026-05-18 11:48:50,831: INFO/ForkPoolWorker-1] [BATCH] Chunk 16/348 processed: 10 orders, revenue=10081.20
```

```
[2026-05-18 11:48:50,978: INFO/ForkPoolWorker-1] [BATCH] Chunk 338/348 processed: 10 orders, revenue=12015.42
[2026-05-18 11:48:50,979: INFO/ForkPoolWorker-1] [BATCH] Chunk 339/348 processed: 10 orders, revenue=11556.96
[2026-05-18 11:48:50,979: INFO/ForkPoolWorker-1] [BATCH] Chunk 340/348 processed: 10 orders, revenue=14609.48
[2026-05-18 11:48:50,980: INFO/ForkPoolWorker-1] [BATCH] Chunk 341/348 processed: 10 orders, revenue=11182.69
[2026-05-18 11:48:50,980: INFO/ForkPoolWorker-1] [BATCH] Chunk 342/348 processed: 10 orders, revenue=12218.04
[2026-05-18 11:48:50,981: INFO/ForkPoolWorker-1] [BATCH] Chunk 343/348 processed: 10 orders, revenue=14299.64
[2026-05-18 11:48:50,981: INFO/ForkPoolWorker-1] [BATCH] Chunk 344/348 processed: 10 orders, revenue=8832.02
[2026-05-18 11:48:50,982: INFO/ForkPoolWorker-1] [BATCH] Chunk 345/348 processed: 10 orders, revenue=13478.43
[2026-05-18 11:48:50,982: INFO/ForkPoolWorker-1] [BATCH] Chunk 346/348 processed: 10 orders, revenue=8897.10
[2026-05-18 11:48:50,983: INFO/ForkPoolWorker-1] [BATCH] Chunk 347/348 processed: 10 orders, revenue=8661.68
[2026-05-18 11:48:50,983: INFO/ForkPoolWorker-1] [BATCH] Chunk 348/348 processed: 7 orders, revenue=10335.39
[2026-05-18 11:48:50,983: INFO/ForkPoolWorker-1] [BATCH] Daily sales for 2026-05-17 COMPLETE - 3477 orders, total_revenue=3817931.64, processed in 348 chunk(s) of 10
[2026-05-18 11:48:50,984: INFO/ForkPoolWorker-1] Task apps.core.tasks.run_daily_sales_batch_task[bf45de5b-d3c0-4e4c-9473-1790a226eb7e] succeeded in 0.16793693699992218s:
{'revenue': 3817931.64, 'chunks': 348, 'chunk_size': 10}
```

7. Requirement 5 - Load Distribution

7.1 Problem

A single Django/Gunicorn application instance can become a bottleneck when many users access the backend at the same time. Gunicorn has a fixed number of workers and threads, so when all handlers are busy, new requests must wait.

A single app instance also concentrates all request pressure on one application container. This creates one performance bottleneck and one point of failure for the HTTP layer.

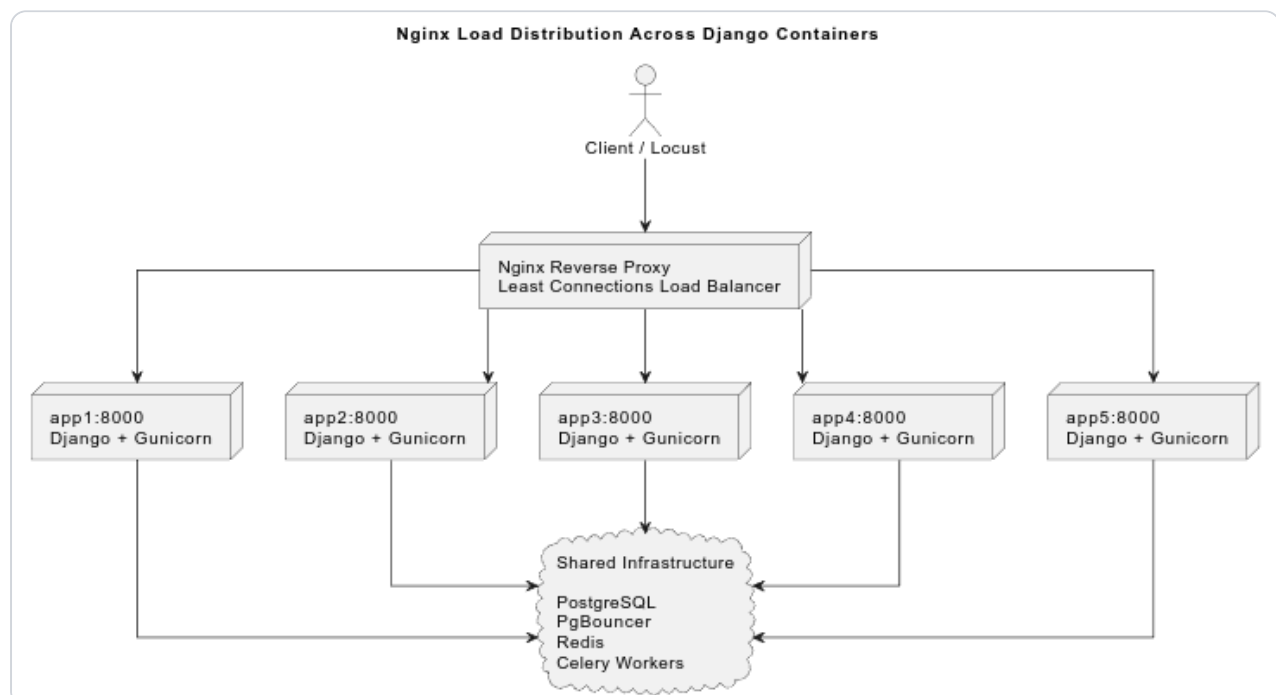
7.2 Solution: Horizontal Scaling with Nginx

The project runs five identical Django/Gunicorn application containers:

```
app1:8000
app2:8000
app3:8000
app4:8000
app5:8000
```

All external HTTP traffic first reaches Nginx. Nginx then forwards each request to one of the backend containers. All app containers share the same PostgreSQL, Redis, PgBouncer, and Celery infrastructure, so any app container can handle any API request.

This gives the backend horizontal scaling at the application layer:



7.3 Load-Balancing Strategy

The active load-balancing algorithm is **Least Connections**:

```
least_conn;
```

Least Connections sends each new request to the backend with the fewest active connections at that moment. This is better than simple Round Robin for this project because the workload is mixed:

- Product reads can be fast, especially when cached.
- Cart operations are usually light.
- Adding to cart can require database writes.
- Checkout is heavier because it uses transactions, stock validation, and row-level locking.

Round Robin does not know whether one backend is already busy with a slow request. Least Connections adapts to current backend activity and reduces the chance that expensive checkout requests pile up on one container while other containers are less busy.

The Nginx upstream configuration also includes basic failure handling:

```
server app1:8000 max_fails=3 fail_timeout=30s;  
server app2:8000 max_fails=3 fail_timeout=30s;  
server app3:8000 max_fails=3 fail_timeout=30s;  
server app4:8000 max_fails=3 fail_timeout=30s;  
server app5:8000 max_fails=3 fail_timeout=30s;
```

This means Nginx can temporarily stop sending traffic to a backend that repeatedly fails.

7.4 Current Capacity

Each application container runs:

```
8 Gunicorn workers  
3 threads per worker
```

Total controlled request capacity:

```
5 app containers x 8 Gunicorn workers x 3 threads = 120 request handlers
```

This does not mean the database can safely process 120 heavy transactional operations at the same time. Requirement 2 still controls database pressure through PgBouncer, Django connection reuse, and bounded worker counts. Requirement 5 focuses on distributing HTTP request handling across multiple app containers.

7.5 Why IP Hash Was Not Used

IP Hash binds the same client IP to the same backend. This is useful for sticky sessions, but this project does not require sticky sessions because token-based authentication is used and the API is stateless.

IP Hash was not selected because:

- The API does not depend on local server state.
- During Locust tests, many requests may come from the same IP or container.
- IP Hash could send too much traffic to one backend and reduce distribution quality.

7.6 Validation Commands

Start the full system:

```
docker compose up --build -d
```

Verify that Nginx and all five app containers are running:

```
docker ps
```

Expected containers:

```
ecommerce_nginx
ecommerce_app1
ecommerce_app2
ecommerce_app3
ecommerce_app4
ecommerce_app5
ecommerce_db
ecommerce_redis
ecommerce_pgouncer
```

Run a normal workload through Nginx:

```
docker compose run --rm \
-e LOCUST_MODE=normal \
-e ADMIN_TOKEN="<admin-token>" \
locust -f locustfile.py --host http://nginx:80 \
--users 50 --spawn-rate 10 --headless --run-time 1m
```

REQUIREMENTS TEST REPORT

Mode: normal

REQ 1 - Race Condition Safety (5 Cases)

[General] HOT checkouts via EcommerceUser succeeded : 0
[General] Stock-outs blocked : 0

REQ 2 - Resource Management & Capacity Control

Total requests : 1044

Errors : 0

Evidence to screenshot:

- Locust response time
 - docker stats
 - pg_stat activity
 - active_queues for Celery workers
 - Docker Compose workers/concurrency
 - Exponential backoff code in services.py
- ✗ 8 unexpected errors

REQ 3 - Async Queues (BEFORE vs AFTER)

AFTER (async Celery): avg=17ms samples=294

REQ 4 - Batch Processing (BEFORE vs AFTER)

i Run with LOCUST_MODE=req4_before or req4_after to test batch processing

First hit (DB) : 35ms

Cache hit (Redis) : 7ms

Speedup : 4.7x ✓

[2026-05-18 14:53:52,871] 1eb7395f4d12/INFO/locust.main: Shutting down (exit code 1)

Type	Name	# reqs	# fails	Avg	Min	Max	Med	req/s	failures/s
GET	/api/cart/ [VIEW]	94	0(0.00%)	5	2	167	3	1.57	0.00
POST	/api/cart/add/	564	8(1.42%)	9	2	246	6	9.42	0.13
DELETE	/api/cart/clear/	297	0(0.00%)	3	2	13	3	4.96	0.00
GET	/api/orders/ [LIST]	120	0(0.00%)	10	2	44	10	2.00	0.00
POST	/api/orders/checkout/ [ASYNC]	294	0(0.00%)	17	8	325	12	4.91	0.00
GET	/api/products/ [CACHE TEST]	1932	0(0.00%)	8	3	278	5	32.27	0.00
GET	/api/products/ [LIST]	512	0(0.00%)	6	3	228	5	8.55	0.00
GET	/api/products/[id]/	118	0(0.00%)	5	2	200	3	1.97	0.00
GET	/api/products/[id]/ [CACHE TEST]	455	0(0.00%)	4	2	170	3	7.60	0.00
POST	/api/users/login/	25	0(0.00%)	471	170	2031	200	0.42	0.00
POST	/api/users/login/ [browse]	25	0(0.00%)	390	163	1956	220	0.42	0.00
Aggregated		4436	8(0.18%)	12	2	2031	5	74.10	0.13

Response time percentiles (approximated)

Response Type	Name	50%	66%	75%	80%	90%	95%	98%	99%	99.9%	99.99%	100%	# reqs
GET	/api/cart/ [VIEW]	3	4	4	4	4	5	11	170	170	170	170	94
POST	/api/cart/add/	6	7	8	8	9	9	15	180	250	250	250	564
DELETE	/api/cart/clear/	3	4	4	4	4	5	6	7	13	13	13	297
GET	/api/orders/ [LIST]	10	12	12	13	15	17	30	35	45	45	45	120
POST	/api/orders/checkout/ [ASYNC]	12	12	13	14	29	49	76	100	330	330	330	294
GET	/api/products/ [CACHE TEST]	5	5	6	6	11	12	23	180	240	280	280	1932
GET	/api/products/ [LIST]	5	5	6	6	11	12	14	22	230	230	230	512
GET	/api/products/[id]/	3	4	4	4	4	4	5	12	200	200	200	118
GET	/api/products/[id]/ [CACHE TEST]	3	4	4	4	4	5	7	170	170	170	170	455
POST	/api/users/login/	200	340	360	1100	1500	1500	2000	2000	2000	2000	2000	25
POST	/api/users/login/ [browse]	220	270	340	640	800	1100	2000	2000	2000	2000	2000	25
Aggregated		5	6	6	9	12	13	87	200	1100	2000	2000	4436

Error report	# occurrences	Error
8	1	POST /api/cart/add/: Cart add failed: 404

Alternatively, run Locust with the web UI:

```
docker compose run --rm -p 8090:8089 \
  -e LOCUST_MODE=normal \
  -e ADMIN_TOKEN="<admin-token>" \
  locust -f locustfile.py --host http://nginx:80
```

Locust UI values:

```
Number of users: 50
Ramp up: 10
Host: http://nginx:80
```

Map app container IPs:

```
docker inspect -f '{{.Name}} {{range .NetworkSettings.Networks}}
{{.IPAddress}}{{end}}' \
  ecommerce_app1 ecommerce_app2 ecommerce_app3 ecommerce_app4
ecommerce_app5
```

```
[salah@Jarvis high-performance-ecommerce-backend]$ docker inspect -f '{{.Name}} {{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' \
ecommerce_app1 ecommerce_app2 ecommerce_app3 ecommerce_app4 ecommerce_app5
ecommerce_app1 172.18.0.10
ecommerce_app2 172.18.0.5
ecommerce_app3 172.18.0.12
ecommerce_app4 172.18.0.8
ecommerce_app5 172.18.0.11
```

Check Nginx upstream logs:

[illegible]

```
docker logs -f ecommerce_app2
docker logs -f ecommerce_app3
docker logs -f ecommerce_app4
docker logs -f ecommerce_app5
```

[illegible]

7.7 Result

Requirement 5 is satisfied because the system distributes requests across five Django/Gunicorn containers using Nginx with the Least Connections algorithm. The Nginx upstream logs show requests reaching all backend containers, proving horizontal traffic distribution.

8. Reproducibility Checklist

Use this sequence before capturing evidence:

```
docker compose up --build -d
docker compose run --rm app1 python manage.py seed_ecommerce --clean
docker ps
```

Evidence to capture:

Requirement	Evidence
Req 1	<code>race1_before</code> and <code>race1_after</code> Locust output for all five cases
Req 1	<code>SELECT ... WHERE stock < 0</code> after safe run returns 0 rows
Req 2	Locust req2 response metrics, <code>docker stats</code> , <code>pg_stat_activity</code>
Req 3	Sync vs async Locust response time and Celery email logs
Req 4	Naive batch log vs chunked batch log
Req 5	Nginx upstream logs showing <code>app1</code> , <code>app2</code> , <code>app3</code> , <code>app4</code> , and <code>app5</code>

9. Limitations

This is a Docker Compose demonstration environment, not a full high-availability production deployment.

Current limitations:

- Nginx is a single reverse proxy service.
- PostgreSQL is a single database service.
- Redis is a single service.

- PgBouncer is a single pooling service.
- The number of application containers is currently five: `app1` , `app2` , `app3` , `app4` , and `app5` .
- Test results depend on the local machine resources.

These limitations do not invalidate the first five requirements because the goal is to demonstrate synchronization, controlled capacity, asynchronous queues, chunked processing, and load distribution.

10. Final Conclusion

The backend satisfies Requirements 1-5 using the current project implementation.

Requirement 1 is covered through five concrete concurrency cases: overselling, wallet double spend, duplicate payment processing, invalid cancellation during payment, and over-reservation. Requirement 2 is covered through bounded Gunicorn concurrency, PgBouncer pooling, Django connection reuse, and fixed Celery worker concurrency. Requirement 3 is covered by moving slow email and wallet-payment work into Celery queues. Requirement 4 is covered by changing daily sales aggregation from a naive all-at-once job to a chunked batch task. Requirement 5 is covered by Nginx Least Connections load balancing across five Django/Gunicorn application containers.